



UNIVERSITAT DE  
BARCELONA

Facultat de Matemàtiques  
i Informàtica

**GRAU DE MATEMÀTIQUES**

**Treball final de grau**

---

# **On Categorical Semantics And Autonomous Theorem Proving**

---

**Autor: Carlos Fernández Lorán**

**Director: Dr. Carles Casacuberta and Dr. Joost J. Joosten**

**Realitzat a: Departament of  
Mathematics and Computer Science**

**Barcelona, May 15, 2026**



# Contents

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>Abstract</b>                                                          | <b>ii</b> |
| <b>1 The Syntax and Semantics of Generalized Algebraic Theories</b>      | <b>3</b>  |
| 1.1 Generalized Algebraic Theories . . . . .                             | 3         |
| 1.2 Formal Definition . . . . .                                          | 5         |
| 1.3 Contexts and Realizations . . . . .                                  | 8         |
| 1.4 Contextual Categories . . . . .                                      | 9         |
| 1.5 Properties of Contextual Categories . . . . .                        | 11        |
| 1.6 Relevance of the Context Category . . . . .                          | 13        |
| 1.7 Problem Definition . . . . .                                         | 16        |
| 1.8 Expansion Algorithm . . . . .                                        | 17        |
| <b>2 The Calculus of Constructions</b>                                   | <b>25</b> |
| 2.1 Syntactic Formulation . . . . .                                      | 25        |
| 2.2 Doctrines of Constructions . . . . .                                 | 29        |
| 2.3 Interpretation of Calculus of Constructions . . . . .                | 32        |
| 2.4 The Term Model of Calculus of Constructions . . . . .                | 35        |
| 2.5 Extending Calculus of Constructions and its Interpretation . . . . . | 38        |
| 2.6 Problem Redefinition . . . . .                                       | 39        |
| 2.7 Expansion Algorithm . . . . .                                        | 41        |
| <b>3 Implementation of the Expansion Algorithm</b>                       | <b>47</b> |
| 3.1 Term generation in Simply Typed Combinatory Logic . . . . .          | 47        |
| 3.2 Graph Neural Networks . . . . .                                      | 51        |
| <b>4 Conclusion</b>                                                      | <b>57</b> |
| <b>Bibliography</b>                                                      | <b>59</b> |

## Abstract

The main goal of this work is to prove the existence of.....

**Notation:** We will work over an algebraically closed field  $K$  of characteristic zero.

# Introduction

The idea of generating mathematical reasoning computationally is one of the most profound challenges in computer science. At the core of this pursuit lies the Curry-Howard correspondence, an isomorphism between logic and computation. Under this paradigm, mathematical propositions are interpreted as types, and the proofs of these propositions are viewed as terms (or programs) inhabiting those types. Consequently, the problem of proving a theorem is reduced to the problem of finding a well-typed term in a formal type-theory.

Modern proof assistants, such as Rocq (formerly Coq) and Lean, leverage this correspondence using rich type theories like the Calculus of Inductive Constructions. While these environments enabled us to formalize complex theories, fully autonomous theorem proving with such rich frameworks remains an incredibly difficult challenge. The primary obstacle is undecidability of type inhabitation in dependent type theories: attempting to exhaustively construct a term of a given type results in a combinatorial explosion of the search space.

Traditionally, term generation has been approached from a purely syntactic perspective. This has more recently been the area of work of some teams using Large Language Models. But generating terms just from the syntax is very difficult to guide heuristically. This thesis proposes a paradigm shift: leveraging the categorical semantics of type theories to reframe autonomous theorem proving as a structural graph exploration problem. By translating the syntax of Generalized Algebraic Theories (GATs) and the Calculus of Constructions (CoC) into categorical structures - namely Contextual Categories and Doctrines of Constructions - we obtain term models that inherently possess a rich, geometric graph structure.

Withing this categorical framework, the problem of finding a proof translates to finding a specific section (a morphism) withing a relativized contextual category. To solve this, we formalized an "Expansion Algorithm" - a deterministic process that exhaustively builds the finite subcategories representing the search space of terms. However, as with syntactic generation, this raw expansion algorithm suffers from severe exponential growth.

To overcome this, we introduce the application of artificial intelligence, specif-

ically Graph Neural Networks (GNNs), to navigate and prune the categorical search space. Considering our finite subcategories that form the known search space as graphs, they are the perfect inputs for message-passing neural networks. By training a GNN to classify nodes in the context graph (deciding which paths to expand, keep or delete), we can effectively guide the expansion algorithm, bypassing the combinatorial explosion and paving the way for efficient, autonomous theorem proving.

## Thesis Outline

This thesis is structured to guide the reader from the abstract categorical foundations to the practical implementation of AI-guided theorem proving:

- **Chapter 1: The Syntax and Semantics of Generalized Algebraic Theories.** We introduce Cartmell's Generalized Algebraic Theories and define their syntax and semantics. We construct the contextual category  $C(U)$ , define the problem of type inhabitation in categorical terms, and present the foundational Expansion Algorithm.
- **Chapter 2: The Calculus of Constructions.** We step up the complexity to the Calculus of Constructions (CoC), introducing product types and the universe of propositions. We study Doctrines of Constructions as models for CoC and extend our categorical problem definition and Expansion Algorithm to handle this richer syntax.
- **Chapter 3: Implementation of the Expansion Algorithm.** We bring the theory into practice. We demonstrate the raw Expansion Algorithm generating terms of Simply Typed Combinatory Logic ( $CL_{\rightarrow}$ ) using `IMPLICIA`, a custom Rust-based graph engine. Finally, we outline the architecture for integrating Graph Neural Networks using novel unsupervised "Conjecturer-Prover" training paradigm to optimize the expansion process.
- **Chapter 4: Conclusion.** We summarize our findings, discuss the implications of working with categorical semantics and neural networks, and compare it with our previous works on syntactical generation with Language Models. Finally we outline future directions of our research.

# Chapter 1

## The Syntax and Semantics of Generalized Algebraic Theories

In this chapter, we introduce Generalized Algebraic Theories (GATs), a framework for working with dependent types. Our ultimate goal in this thesis is to translate the syntactical problem of theorem proving into a geometrical graph-exploration problem. GATs allow us to construct the concept of *contextual category*, the foundational categorical structure to deal with dependent types. We will use this structure as a baseline for interpreting the more complex Calculus of Constructions in Chapter 2.

### 1.1 Generalized Algebraic Theories

Generalized Algebraic Theories, originally presented by J. Cartmell in his thesis [2] [3], is an abstraction of Martin-Löf type theory and is a generalization of the usual notion of a many-sorted algebraic theory in the following manner: while in Martin Löff's many-sorted algebraic theories sorts are constant types, interpreted as sets, in generalized algebraic theories sorts might be constant or variable, over a specified range of types. Variable types are interpreted as families of sets.

Formally, a generalized algebraic theory consists of:

1. a set of sorts, that can be constant or variable;
2. a set of operators, with the sort of the arguments and value specified;
3. a set of axioms, where each axiom is an identity between similar expressions.

To build intuition, consider the theory of categories. The set of sorts contains two elements *Ob* and *Hom*. *Ob* is a constant type and *Hom* is a variable type; for

any two terms  $x, y$  of type  $Ob$ , we have a type  $Hom(x, y)$ . Here, we see a key feature of GATs: types vary over terms, not over types. The set of operators also has two elements  $id$  and  $\circ$ . The  $id$  operator has one argument  $x$  of type  $Ob$ , and returns a value of type  $Hom(x, x)$ . This shows how the type of value of an operator might vary depending on the arguments. The  $\circ$  operator (or composition) expects 5 arguments, three terms  $x, y, z$  of type  $Ob$ , one term  $f$  of type  $Hom(x, y)$ , and one term  $g$  of type  $Hom(y, z)$ . It returns a value of type  $Hom(x, z)$ . Here we see that the type of an argument might also vary depending on a previous argument. Informally, instead of writing  $\circ(x, y, z, f, g)$  we can write  $\circ(f, g)$ , as the first three arguments might be inferred from the last two. This notation is discussed in the original paper [3].

Now we will introduce the notation we will be using when talking about terms and types. This notation will be formally defined in a later section. So consider you have a set of variables  $V$ , we will associate terms to variables as required. When asserting that a term  $t$  is of type  $A$  we will write  $t : A$ , but if  $t$  has variables  $x_1, \dots, x_n$  occurring in it this assertion does not make sense unless we assign some types to  $x_1, \dots, x_n$ . To do this we will write  $x_1 : A_1, \dots, x_n : A_n \vdash t : A$  or

$$\frac{x_1 : A_1, \dots, x_n : A_n}{t : A}$$

that reads as: if  $x_1$  is a variable of type  $A_1$ , ... and if  $x_n$  is a variable of type  $A_n$ , then  $t$  is a term of type  $A$ . Similarly, if we want to assert that if  $x_1$  is a variable of type  $A_1$ , ... and if  $x_n$  is a variable of type  $A_n$ , then  $A$  is a type, we can write:  $x_1 : A_1, \dots, x_n : A_n \vdash A \text{ type}$  or

$$\frac{x_1 : A_1, \dots, x_n : A_n}{t : A}$$

this assertions are called rules or sequents. Rules are used to express which expressions are well-formed. We will work with rules as units instead of the base expressions. This is because terms and types do not make sense by themselves but they need to be thought of in a context.

Axioms are also written as rules. The conclusion of those rules are equalities between types or between terms of the same type. We write those rules as:

$$\frac{x_1 : A_1, \dots, x_n : A_n}{A = A'} \qquad \frac{x_1 : A_1, \dots, x_n : A_n}{t = t' : A}$$

and read as: if  $x_1$  is a variable of type  $\Delta_1$ , ... and  $x_n$  is a variable of type  $\Delta_n$ , then  $\Delta = \Delta'$ , or  $t = t'$  as terms of type  $\Delta$ , respectively. We will be using both shorthand and fraction notation as needed.

To continue with the previous example, we can write the axioms of category



theory as:

$$\begin{aligned} x, y : Ob, f : Hom(x, y), &\vdash \circ(id(x), f) = f \\ x, y : Ob, f : Hom(x, y), &\vdash \circ(f, id(y)) = f \\ w, x, y, z : Ob, f : Hom(w, x), g : Hom(x, y), h : Hom(y, z) \\ &\vdash \circ(\circ(f, g), h) = \circ(f, \circ(g, h)) \end{aligned}$$

When presenting a theory's language i.e. the sets of sorts and operators, we must present each symbol with a rules specifying the role of this symbol, either as a sort symbol or as specifically typed operator symbol. This rules are called introductory rules. In the case of a sort symbol  $A$  the rule should be like

$$\frac{x_1 : A_1, \dots, x_n : A_n}{A(x_1, \dots, x_n) \text{ type}}$$

clearly specifying the types  $A$  is dependent on. In the case of an operator symbol, a rule of the form

$$\frac{x_1 : A_1, \dots, x_n : A_n}{f(x_1, \dots, x_n) : A}$$

suffices to specify the types of the arguments and the value of  $f$ .

Finally, to present a theory we will give a set of symbols, each associated to its introductory rule, and a set of axioms. And, of course, everything must be well-formed, **something we will address in the following section**. So now we can give a formal presentation of the theory of categories:

| <i>Symbol</i>                                    | <i>Introductory Rule</i>                                                                                               |
|--------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Ob                                               | Ob is a type.                                                                                                          |
| Hom                                              | $x, y \in \text{Ob} : \text{Hom}(x, y)$ is a type.                                                                     |
| $\circ$                                          | $x, y, z \in \text{Ob}, f \in \text{Hom}(x, y), g \in \text{Hom}(y, z) : \circ(f, g) \in \text{Hom}(x, z).$            |
| id                                               | $x \in \text{Ob} : \text{id}(x) \in \text{Hom}(x, x).$                                                                 |
| <i>Axioms</i>                                    |                                                                                                                        |
| $\circ(\text{id}(x), f) = f,$                    | whenever $x, y \in \text{Ob}$ and $f \in \text{Hom}(x, y).$                                                            |
| $\circ(f, \text{id}(y)) = f,$                    | whenever $x, y \in \text{Ob}$ and $f \in \text{Hom}(x, y).$                                                            |
| $\circ(\circ(f, g), h) = \circ(f, \circ(g, h)),$ | whenever $w, x, y, z \in \text{Ob}, f \in \text{Hom}(w, x),$<br>$g \in \text{Hom}(x, y)$ and $h \in \text{Hom}(y, z).$ |

## 1.2 Formal Definition

In this section we will formally define all of the concepts we just introduced. To give a notion of well-formed introductory rule, we need the concept of drivability,

which also depends on the introductory rules. To tackle this problem we will leave aside the question of well-formedness until we have the complete set of derived rules of a theory. We will first define a pretheory and then define a theory as a well-formed pretheory.

We will assume to have countable set of variables  $V$ . Lets define the notion of rule on an alphabet  $W$ , the elements of  $W$  are called symbols.

**Definition 1.1.** *The set of **expressions** is defined inductively such that expressions are finite sequences of elements of  $W \cup V \cup \{(\} \cup \{)\}$  defined by the clauses:*

1. *if  $x \in V$ , then  $x$  is an expression;*
2. *if  $L \in W$ , then  $L$  is an expression;*
3. *if  $L \in W$  and  $e_1, \dots, e_n$  are expressions, then  $L(e_1, \dots, e_n)$  is an expression.*

Now we can define the building blocks of a rule:

**Definition 1.2.** *A **premise** is defined to be a sequence of  $V \times$  the set of expressions. The empty sequence is allowed and we call it an empty premise. If  $((x_1, A_1), \dots, (x_n, A_n))$  is a premise, we write it as  $x_1 : A_1, \dots, x_n : A_n$ .*

**Definition 1.3.** *We call a **conclusion** to one of the following:*

1. *T-conclusion: defined by a single expression  $A$  and written as ' $A$  is a type'.*
2.  *$\in$ -conclusion: defined by a pair of expressions  $(t, A)$  and written as ' $t : A$ '.*
3.  *$T =$ conclusion: defined by a pair of expressions  $(A_1, A_2)$  and written as ' $A_1 = A_2$ '.*
4.  *$\in =$ conclusion: defined by a triple of expressions  $(t_1, t_2, A)$  and written as ' $t_1 = t_2 : A$ '.*

Finally a **rule** is determined by a premise  $P$  and a conclusion  $C$ , and is written as  $\frac{P}{C}$  or as  $P \vdash C$ . We name the rule as  $T$ -rule,  $\in$ -rule,  $T =$ rule or  $\in =$ rule according to the form of the conclusion.

Now we can define pretheory and later theory.

**Definition 1.4.** *A **pretheory** consists of*

1. *a set of sort symbols  $S$ ;*
2. *a set of operator symbols  $Z$ ;*

3. for each sort symbol  $A$ , an associated rule of the alphabet  $S \cup Z$ :

$$\frac{x_1 : A_1, \dots, x_n : A_n}{A(x_1, \dots, x_n) \text{ type}}$$

for some  $n \geq 0$ . It is called the introductory rule of  $A$ ;

4. for each operator symbol  $F$ , an associated rule of the alphabet  $S \cup Z$ :

$$\frac{x_1 : A_1, \dots, x_n : A_n}{F(x_1, \dots, x_n) : A}$$

for some  $n \geq 0$ . It is called the introductory rule of  $F$ ;

5. a set of axioms. Each axiom is either a  $T = \text{rule}$  or a  $\in = \text{rule}$  of the alphabet  $S \cup Z$ .

Now we will tackle the problem of well-formedness of rules.

**Definition 1.5.** If  $U$  is a pretheory, then

1. a **context** is a premise  $x_1 : \Delta_1, \dots, x_n : \Delta_n$  such that the rule

$$\frac{x_1 : \Delta_1, \dots, x_{n-1} : \Delta_{n-1}}{\Delta_n \text{ is a type}}$$

is a derived rule such that  $x_n$  is distinct from all of  $x_1, \dots, x_{n-1}$ .

2. (a) the rule  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{\Delta \text{ is a type}}$  is **well-formed** if the premise  $x_1 : \Delta_1, \dots, x_n : \Delta_n$  is a context.
- (b) the rule  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{t : \Delta}$  is **well-formed** if  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{\Delta \text{ is a type}}$  is a derived rule of  $U$ .
- (c) the rule  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{\Delta = \Delta'}$  is **well-formed** if  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{\Delta \text{ is a type}}$  and  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{\Delta' \text{ is a type}}$  are derived rules of  $U$ .
- (d) the rule  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{t = t' : \Delta}$  is **well-formed** if  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{t : \Delta}$  and  $\frac{x_1 : \Delta_1, \dots, x_n : \Delta_n}{t' : \Delta}$  are derived rules of  $U$ .

**Definition 1.6.** The set of **derived rules** of a pretheory  $U$  is the set of rules derivable by the following principles of derivation:

**Definition 1.7.** A pretheory is **well-formed** if all of its introductory rules are well-formed. A **theory** is a well-formed pretheory.

Lastly, we will introduce substitution. Given  $\Delta, t_1, \dots, t_n$  expressions and  $x_1, \dots, x_n$  variables, then we define the expression  $\Delta[x_1|t_1, \dots, x_n|t_n]$  the result of simultaneous replacement of  $x_1, \dots, x_n$  in  $\Delta$  by  $t_1, \dots, t_n$ . We call this the Substitution operation. Then, as proven in the original paper [3], the set of derived rules of a theory is closed under the operation of substitution of correctly typed terms for variables. We could add substitution as one of the derivation rules but would make applying induction very messy.

### 1.3 Contexts and Realizations

To transition from syntax to geometry (which our Graph Neural Network will eventually navigate), we construct a category out of the theory syntax.

Let  $U$  be a theory, we define the category of contexts and realizations, denoted  $R(U)$ . This category has as objects the contexts of  $U$ . Given two contexts  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n$  and  $\Delta \equiv y_1 : B_1, \dots, y_m : B_m$  of  $U$ , a morphism from  $\Gamma$  to  $\Delta$ , or a realization of  $\Delta$  from  $\Gamma$ , is a  $m$ -tuple  $\langle t_1, \dots, t_m \rangle$  such that for each  $j$ ,  $1 \leq j \leq m$ , the rule

$$\Gamma \vdash t_j : B_j[y_1 \mid t_1, \dots, y_{j-1} \mid t_{j-1}]$$

is derivable. The identity of an object  $x_1 : \Delta_1, \dots, x_n : \Delta_n$  is the realization  $\langle x_1, \dots, x_n \rangle$ , and composition is defined as

$$\begin{aligned} \langle t_1, \dots, t_m \rangle \circ \langle s_1, \dots, s_w \rangle = \\ \langle s_1[y_1 \mid t_1, \dots, y_m \mid t_m], \dots, s_w[y_1 \mid t_1, \dots, y_m \mid t_m] \rangle \end{aligned}$$

whenever  $\Gamma = x_1 : A_1, \dots, x_n : A_n$ ,  $\Delta = y_1 : B_1, \dots, y_m : B_m$  and  $\Omega = z_1 : C_1, \dots, z_w : C_w$  are contexts, and  $\langle t_1, \dots, t_m \rangle : \Gamma \rightarrow \Delta$  and  $\langle s_1, \dots, s_w \rangle : \Delta \rightarrow \Omega$  are realizations.

Now we will study some properties of this category.

First, we will see that it has a tree structure.

**Definition 1.8.** A tree is a pair  $(N, \text{father})$  where  $N$  is a not empty set of nodes and  $\text{father} : N \rightarrow N$  such that:

1. for all  $A \in N$  there exists  $n \in \omega$  such that  $\text{father}^n(A) = \text{father}^{n+1}(A)$ ;
2. for all  $A, B \in N$ , if  $A = \text{father}(A)$  and  $B = \text{father}(B)$ , then  $A = B$ .

We will write  $A \sqsubseteq B$  if  $A = \text{father}(B)$  and  $A \triangleleft B$  if  $A \sqsubseteq B$  and  $A \neq B$ . By condition 2, there is a unique least element of  $N$ , call it 1. Finally we will define, for any  $A \in N$ ,  $\text{level}(A)$  to be the least  $n \in \omega$  with  $\text{father}^n(A) = \text{father}^{n+1}(A)$ . We can see that the set of objects of  $R(U)$ , i.e. the set of contexts of  $U$  is structured as a tree with the empty context as it's least element and  $\text{father}(\langle x_1 : \Delta_1, \dots, x_n : \Delta_n \rangle) =$

$\langle x_1 : \Delta_1, \dots, \Delta_{n-1} \rangle$ . Moreover, given a context  $x_1 : A_1, \dots, x_n : A_n, x_{n+1} : A_{n+1}, \dots, x_{n+m} : A_{n+m}$  of  $U$ ,  $\langle x_1, \dots, x_n \rangle$  is a realization of  $x_1 : A_1, \dots, x_n : A_n$  with regard to  $x_1 : A_1, \dots, x_n : A_n, x_{n+1} : A_{n+1}, \dots, x_{n+m} : A_{n+m}$  and is noted as  $p(\langle x_1 : A_1, \dots, x_{n+m} : A_{n+m} \rangle, \langle x_1 : A_1, \dots, x_n : A_n \rangle)$ . Hence, given two objects  $\Gamma, \Delta \in \text{Ob}R(U)$ , such that  $\Gamma \leq \Delta$ ,  $p(\Delta, \Gamma)$  is defined and  $p(\Delta, \Gamma) : \Delta \rightarrow \Gamma$  in  $R(U)$ .

And if  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n$ ,  $\Gamma' \equiv y_1 : B_1, \dots, y_m : B_m$  and  $\Delta \equiv y_1 : B_1, \dots, y_{m+w} : B_{m+w}$  are contexts, and if  $f = \langle t_1, \dots, t_m \rangle : \Gamma \rightarrow \Gamma'$  in  $R(U)$ , then

$$\begin{array}{ccc} f^* \Delta & \xrightarrow{q(f, \Delta)} & \Delta \\ p(f^* \Delta, \Gamma) \downarrow & & \downarrow p(\Delta, \Gamma') \\ \Gamma & \xrightarrow{f} & \Gamma' \end{array}$$

is a pullback diagram in  $R(U)$ , where

$$\begin{aligned} f^* \Delta &\equiv x_1 : A_1, \dots, x_n : A_n, \\ &\quad y_{m+1} : B[y_1 \mid t_1, \dots, y_m \mid t_m], \dots, y_{m+w} : B[y_1 \mid t_1, \dots, y_m \mid t_m] \\ q(f, \Delta) &= \langle t_1, \dots, t_m, y_{m+1}, \dots, y_{m+w} \rangle \end{aligned}$$

This motivates the following definition

## 1.4 Contextual Categories

**Definition 1.9.** *a contextual category consists of*

1. a category  $\mathcal{C}$  with a terminal object  $1$ ;
2. a function  $\text{father} : \text{Ob}\mathcal{C} \rightarrow \text{Ob}\mathcal{C}$  such that  $(\text{Ob}\mathcal{C}, \text{father})$  is a tree, and  $1$  is the root of the tree, i.e.  $\text{father}1 = 1$ ;
3. for each object  $A$  of  $\mathcal{C}$  different from  $1$ , a morphism  $p(A) : A \rightarrow \text{father}A$  called canonical projection of  $A$ , we write it as  $A \twoheadrightarrow \text{father}(A)$ ;
4. for all objects  $A, B$  in  $\mathcal{C}$  with  $A \triangleleft B$  and all morphisms  $f : C \rightarrow A$ , an object  $f^*B$  of  $\mathcal{C}$  such that  $C \triangleleft f^*B$ , and a morphism  $q(f, B) : f^*B \rightarrow B$  in  $\mathcal{C}$  such that

$$\begin{array}{ccc} f^*B & \xrightarrow{q(f, B)} & B \\ \downarrow & & \downarrow \\ C & \xrightarrow{f} & A \end{array}$$

is a pullback in  $\mathcal{C}$  that satisfies the following functoriality conditions:

(a) if  $A$  and  $B$  are objects in  $\mathcal{C}$  such that  $A \triangleleft B$ , then

$$id_A^* B = B \quad \text{and} \quad q(id_A, B) = id_B$$

(b) whenever

$$\begin{array}{ccccc} & & & & B \\ & & & & \downarrow \\ A & \xrightarrow{f} & A' & \xrightarrow{f'} & A'' \end{array}$$

in  $\mathcal{C}$ , then

$$(f' \circ f)^* B = f^* (f'^* B) \quad \text{and} \quad q(f' \circ f, B) = q(f', B) \circ q(f, f'^* B)$$

We have that  $R(U)$  is structured as a contextual category. The problem with  $R(U)$  is that semantically equal but syntactically different rules are interpreted as different elements. Hence we will define an equivalence relation between derived  $T$ - and  $\in$ - rules such that:

$$x_1 : A_1, \dots, x_n : A_n \vdash A_{n+1} \text{type} \equiv y_1 : B_1, \dots, y_m : B_m \vdash B_{m+1} \text{type}$$

if, and only if,  $m = n$  and for each  $1 \leq i \leq n + 1$ ,

$$x_1 : A_1, \dots, x_{i-1} : A_{i-1} \vdash A_i = B_i[y_1 \mid x_1, \dots, y_{i-1} \mid x_{i-1}]$$

is derivable, and

$$x_1 : A_1, \dots, x_n : A_n \vdash t : A \equiv y_1 : B_1, \dots, y_m : B_m \vdash s : B$$

if, and only if,

$$x_1 : A_1, \dots, x_n : A_n \vdash A \text{type} \equiv y_1 : B_1, \dots, y_m : B_m \vdash B \text{type}$$

and

$$x_1 : A_1, \dots, x_n : A_n \vdash t = s[y_1 \mid x_1, \dots, y_m \mid x_m] : A$$

is derivable.

We define  $C(U)$  as the homomorphic image of  $R(U)$  induced by this equivalence relation. This means that  $C(U)$  is the category of equivalence classes of contexts and of equivalence classes of realizations. Two contexts  $x_1 : A_1, \dots, x_n : A_n$  and  $y_1 : B_1, \dots, y_m : B_m$  are equivalent if, and only if,

$$x_1 : A_1, \dots, x_{n-1} : A_{n-1} \vdash A_n \text{type} \equiv y_1 : B_1, \dots, y_{m-1} : B_{m-1} \vdash B_m \text{type}$$

and two realizations

$$\begin{aligned} \langle t_1, \dots, t_m \rangle : \langle x_1 : A_1, \dots, x_n : A_n \rangle &\rightarrow \langle y_1 : B_1, \dots, y_m : B_m \rangle \\ \langle s_1, \dots, s_m \rangle : \langle x'_1 : A'_1, \dots, x'_n : A'_n \rangle &\rightarrow \langle y'_1 : B'_1, \dots, y'_m : B'_m \rangle \end{aligned}$$

are equivalent if, and only if, for each  $j$ ,  $1 \leq j \leq n$ ,

$$\begin{aligned} x_1 : A_1, \dots, x_n : A_n \vdash t_j : B_j[y_1 \mid t_1, \dots, y_{j-1} \mid t_{j-1}] &\equiv \\ x'_1 : A'_1, \dots, x'_n : A'_n \vdash s_j : B'_j[y'_1 \mid s_1, \dots, y'_{j-1} \mid s_{j-1}] & \end{aligned}$$

It is easy to see that  $C(U)$  is itself a contextual category.

**Definition 1.10.** *if  $\mathcal{C}$  and  $\mathcal{C}'$  are contextual categories, then a **contextual functor**  $F : \mathcal{C} \rightarrow \mathcal{C}'$  is a functor  $F : \mathcal{C} \rightarrow \mathcal{C}'$  such that:*

1.  $F(1) = 1$  and if  $A \triangleleft B$  in  $\mathcal{C}$ , then  $F(A) \triangleleft F(B)$  in  $\mathcal{C}'$ ;
2. for all object  $A$  of  $\mathcal{C}$ ,  $F(p(A)) = p(F(A))$ ;
3. for all  $f$  and  $B$  such that  $f^*B$  is defined in  $\mathcal{C}$ ,

$$F(f^*B) = F(f)^*F(B) \quad \text{and} \quad F(q(f, B)) = q(F(f), F(B))$$

**Con** is the category of contextual categories and contextual functors.

## 1.5 Properties of Contextual Categories

In this section we will state some properties about contextual categories that will be used later.

The first useful result is about the projections, although we defined projections on pairs  $A \triangleleft B$ , we might extend the notion to pairs  $A \leq B$  defined as

$$p(B, A) = p(X_1) \circ \dots \circ p(X_n) \circ p(B)$$

where  $X_1, \dots, X_n$  is the unique sequence of objects in  $\mathcal{C}$  such that  $A \triangleleft X_1 \triangleleft \dots \triangleleft X_n \triangleleft B$  in  $\mathcal{C}$ . In the case where  $A = B$ , we will take  $p(B, A) = \text{id}_A$ . Notation:  $B \twoheadrightarrow A$ .

In type theory, proving a theorem often requires working under a set of assumptions (a context). In categorical semantics, "assuming a context  $A$ " corresponds to taking the relativization of  $\mathcal{C}$  to  $A$ , noted as  $\mathcal{C}_A$ . More formally, let  $\mathcal{C}$  be a contextual category and  $A$  an arbitrary object of  $\mathcal{C}$ . Then we define the **relativization** of  $\mathcal{C}$  to  $A$ , as the contextual category where:

1. the objects of  $\mathcal{C}_A$  are the objects  $B$  of  $\mathcal{C}$  such that  $A \leq B$ .
2. for objects  $B, C$  in  $\mathcal{C}_A$ , a morphism from  $B$  to  $C$  over  $A$  is a morphism, i.e. a morphism from  $B$  to  $C$  in  $\mathcal{A}$ , is a morphism  $f: B \rightarrow C$  in  $\mathcal{C}$  such that  $p(C, A) \circ f = p(B, A)$ . Composition of morphisms is inherited from  $\mathcal{C}$ .
3.  $\text{father}_A(B) = \text{father}(B)$  if  $A < B$ , and  $\text{father}_A(A) = A$ .
4. for objects  $B > A$ ,  $p_A(B) = p(B): B \rightarrow \text{father}(B)$ .
5. for  $B, C, D$  objects of  $\mathcal{C}_A$  with  $\text{father}_A(D) = C$  and morphisms  $f: B \rightarrow C$  we put

$$f^*A D = f^*D \quad \text{and} \quad q_A(f, D) = q(f, D)$$

Now we will see that any morphism  $f: B \rightarrow A$  in  $\mathcal{C}$  induces a pullback functor  $f^*: \mathcal{C}_A \rightarrow \mathcal{C}_B$  which is a contextual functor. This functor can be thought as a way of generalizing knowledge we have in one part of the category to the rest of the category.

**Lemma 1.11.** *let  $\mathcal{C}$  be a contextual category, and  $f: B \rightarrow A$  a morphism in  $\mathcal{C}$ , then for any object  $C$  with  $C \geq A$  we define an object  $f^*C \geq B$  and a morphism  $q(f, C): f^*C \rightarrow C$  by induction over  $\text{level } C - \text{level } A$ :*

1. if  $C = A$ ,

$$f^*C = B \quad \text{and} \quad q(f, C) = f$$

2. if  $C > A$ ,

$$f^*C = q(f, \text{father}(C))^*C \quad \text{and} \quad q(f, C) = q(q(f, \text{father}(C)), C)$$

then if  $C \geq A$ , the pair  $(p(f^*C, B), q(f, C))$  is a pullback cone over  $(f, p(C, A))$ .

**Theorem 1.12.** *Let  $\mathcal{C}$  be a contextual category and  $f: B \rightarrow A$  a morphism in  $\mathcal{C}$ , by mapping an object  $C$  of  $\mathcal{C}_A$  to  $f^*C$  in  $\mathcal{C}_B$ , and mapping  $g: C \rightarrow D$  over  $A$  to  $f^*g: f^*C \rightarrow f^*D$  over  $B$  determined by*

$$\begin{aligned} p(f^*D, B) \circ f^*g &= p(f^*C, B) \\ q(f, D) \circ f^*g &= q(f, C) \end{aligned}$$

we obtain the contextual functor  $f^*: \mathcal{C}_A \rightarrow \mathcal{C}_B$  called *reindexing along  $f$* .



**Theorem 1.13.** *Let  $\mathcal{C}$  be a contextual category and  $f: B \rightarrow A$ ,  $g: C \rightarrow B$  be morphisms in  $\mathcal{C}$ . Then,*

$$g^* \circ f^* = (f \circ g)^* \quad \text{and} \quad q(f \circ g, D) = q(f, D) \circ q(g, f^*D)$$

for objects  $D \geq A$ .

**Definition 1.14.** *Let  $\mathcal{C}$  be a contextual category,  $A$  and  $B$  be objects of  $\mathcal{C}$  such that  $A \triangleleft B$ , the set of **sections** of  $B$  is defined as*

$$\text{Sections}(B) := \{s: A \rightarrow B \mid p(B) \circ s = \text{id}_A\}$$

If  $\text{level}(B) = 1$ , then

$$\text{Sections}(B) = \{s \mid s: 1 \rightarrow B\}$$

as any morphism  $s: 1 \rightarrow B$  satisfies  $p(B) \circ s = \text{id}_1$  as  $1$  is a terminal object. In this case, the notion of section matches the notion of *global element* of  $B$ . The concept of section of a context  $\langle x_1: A_1, \dots, x_n: A_n, y: B \rangle$  is the categorical equivalent of a term of type  $B$  under context  $\langle x_1: A_1, \dots, x_n: A_n \rangle$ .

## 1.6 Relevance of the Context Category

The context category is the first example of what it is called a **term model**, this means that it is a category built directly from the syntax of a theory without picking any particular mathematical interpretation. This gives you the *most generic* model of that theory. The three main properties of this models are:

1. **Soundness:** any model of such theory corresponds to a structure preserving functor out of the term model.
2. **Completeness:** a judgement is provable/derivable in the theory if it holds in all models, or equivalently if it holds in the term model.
3. **Initiality:** the term model is often the initial object of the category of all models.

Because of Completeness, navigating  $C(U)$  is equivalent as manipulating the syntax of  $U$ .

We will start by seeing the completeness of  $C(U)$ . The following completeness theorem is direct from the definition of  $C(U)$ :

**Theorem 1.15** (Completeness Theorem). *Let  $\mathcal{M}$  be the canonical interpretation of a theory  $U$  in  $C(U)$ .*

1. the  $T$ -rule

$$\frac{x_1 : A_1, \dots, x_n : A_n}{A \text{ type}}$$

is derivable if, and only if,  $x_1 : A_1, \dots, x_n : A_n, x : A$  is a context, i.e. if  $[x_1 : A_1, \dots, x_n : A_n, x : A]$  in  $C(U)$ .

2. the  $\in$ -rule

$$\frac{x_1 : A_1, \dots, x_n : A_n}{t : A}$$

is derivable if, and only if,  $\langle x_1, \dots, x_n, t \rangle$  is a section of  $[x_1 : A_1, \dots, x_n : A_n, x : A]$  in  $C(U)$ .

3. if the following  $T$ -rules are derivable:

$$\Gamma \vdash A_1 \text{ type} \quad \text{and} \quad \Gamma \vdash A_2 \text{ type}$$

Then the rule  $\Gamma \vdash A_1 = A_2$  is derivable if, and only if,

$$[\Gamma, x : A_1] = [\Gamma, x : A_2]$$

where  $x$  is a variable that does not appear on  $\Gamma$ .

4. if the following  $\in$ -rules are derivable:

$$\Gamma \vdash t : A \quad \text{and} \quad \Gamma \vdash s : A$$

Then the rule  $\Gamma \vdash t = s : A$  is derivable if, and only if,  $\langle x_1, \dots, x_n, t \rangle$  and  $\langle x_1, \dots, x_n, s \rangle$  are equal sections of  $[\Gamma, x : A]$  in  $C(U)$ , where  $x$  is a variable not appearing in  $\Gamma$ .

Now we will prove the initiality of  $C(U)$ . Hence, we need to define interpretation for a theory  $U$  on a contextual category, and see that the interpretation into  $C(U)$  is the initial object of the category of interpretations of  $U$ .

Given a contextual category  $\mathcal{C}$  and a theory  $U$ , an **interpretation** of  $U$  in  $\mathcal{C}$  is a mapping  $\mathcal{M}$  from the  $T$ - and  $\in$ -rules of  $U$  to the  $\mathcal{C}$  such that

1. if  $R$  is a derived  $T$ -rule  $\frac{x_1 : A_1, \dots, x_n : A_n}{A \text{ is a type}}$  and we define rules

$R_i \equiv \frac{x_1 : A_1, \dots, x_{i-1} : A_{i-1}}{A_i \text{ is a type}}$  for  $1 \leq i \leq n$ , then  $\mathcal{M}[R_1], \dots, \mathcal{M}[R_n], \mathcal{M}[R]$  are objects of  $\mathcal{C}$  such that

$$1 \triangleleft \mathcal{M}[R_1] \triangleleft \dots \triangleleft \mathcal{M}[R_n] \triangleleft \mathcal{M}[R]$$

in  $\mathcal{C}$ .

2. if  $R_t$  is a derived  $\in$ -rule  $\frac{x_1: A_1, \dots, x_n: A_n}{t: A}$  then  $\mathcal{M}[R_t]$  is section of  $\mathcal{M}[R]$ .

3. if  $\Gamma \Rightarrow A = A'$  is a derived  $T$ -rule, then if

$$R \equiv \Gamma \Rightarrow A \text{ type} \quad \text{and} \quad R' \equiv \Gamma \Rightarrow A' \text{ type}$$

then  $\mathcal{M}[R] = \mathcal{M}[R']$ .

4. if  $\Gamma \Rightarrow t = t': A$  is a derived  $\in$ -rule, then if

$$R_t \equiv \Gamma \Rightarrow t: A, \quad R_{t'} \equiv \Gamma \Rightarrow t': A, \quad R \equiv \Gamma \Rightarrow A \text{ type}$$

then, both  $\mathcal{M}[R_t]$  and  $\mathcal{M}[R_{t'}]$  are sections of  $\mathcal{R}$  and  $\mathcal{M}[R_t] = \mathcal{M}[R_{t'}]$

There is a canonical interpretation  $\mathcal{M}$  of  $U$  in  $\mathcal{C}(U)$ , defined as follows:

1. if  $R \equiv x_1: A_1, \dots, x_n: A_n \Rightarrow A \text{ type}$  is a derived  $T$ -judgement, then we define

$$\mathcal{M}[R] = [x_1: A_1, \dots, x_n: A_n, x_{n+1}: A]$$

2. if  $R_t \equiv x_1: A_1, \dots, x_n: A_n \Rightarrow t: A$  is a derived  $\in$ -judgement, then we define

$$\mathcal{M}[R_t] = [\langle x_1, \dots, x_n, t \rangle]$$

a section of  $\mathcal{M}[R]$ .

The completeness theorem 1.15 guarantees that  $\mathcal{M}$  is well defined.

Now we shall define the concept of homomorphism between interpretations of a theory  $U$ . Let  $\mathcal{C}, \mathcal{C}'$  be two contextual categories, and let  $\mathcal{M}_{\mathcal{C}}$  and  $\mathcal{M}_{\mathcal{C}'}$  be two  $U$ -interpretations in  $\mathcal{C}$  and  $\mathcal{C}'$  respectively. Then, an interpretation homomorphism is a contextual functor  $F: \mathcal{C} \rightarrow \mathcal{C}'$  such that:

1. if  $R$  is a  $T$ -rule, then  $F\mathcal{M}_{\mathcal{C}}[R] = \mathcal{M}_{\mathcal{C}'}[R]$
2. if  $R_t$  is a  $\in$ -rule, then  $F\mathcal{M}_{\mathcal{C}}[R_t] = \mathcal{M}_{\mathcal{C}'}[R_t]$

So we can define the category  $\mathbf{Con}_U$  as the category with pairs of contextual categories and  $U$ -interpretations and their Homomorphisms. We can easily see that  $(\mathcal{C}(U), \mathcal{M})$  is an initial object of  $\mathbf{Con}_U$ , consider  $(\mathcal{C}, \mathcal{N})$  an object of  $\mathbf{Con}_U$ , let's construct a morphism  $F: ((\mathcal{C}(U), \mathcal{M})) \rightarrow (\mathcal{C}, \mathcal{N})$ , i.e. we have to construct a contextual functor  $F: \mathcal{C}(U) \rightarrow \mathcal{C}$  that *commutes* with the interpretation functions. Consider an object  $[\Gamma]$  of  $\mathcal{C}(U)$ , there is a  $T$ -rule  $R^\Gamma$  such that  $\mathcal{M}[R^\Gamma] = \Gamma$ , we define  $F\Gamma = \mathcal{N}[R^\Gamma]$ . In order to define  $F$  on the morphisms of  $\mathcal{C}(U)$ , it suffices to define it for the sections of  $\mathcal{C}(U)$ , as any morphism

$$\langle t_1, \dots, t_m \rangle: \langle x_1: A_1, \dots, x_n: A_n \rangle \rightarrow \langle y_1: B_1, \dots, y_m: B_m \rangle$$

can be build by composition and  $*$  of sections and projections, and the latter is uniquely determined by the condition that  $F$  is a contextual functor. Hence, given a section  $s = \langle x_1, \dots, x_n, t \rangle$  of  $R \equiv x_1 : A_1, \dots, x_n : A_n, x : A$ , by definition there exists a derived  $\in$ -rule  $R_t$  such that  $\mathcal{M}[R_t] = s$ , so we can define  $Fs = \mathcal{N}[R_t]$ . The uniqueness of  $F$  is rather simple to see, as we cannot assign any other values to any object or morphism of  $C(P)$  without conflicting with the structural conditions of the functor. Hence,  $(C(U), \mathcal{M})$  is an initial object of  $\mathbf{Con}_U$ .

Soundness comes from the initiality of  $C(U)$ , as any model of  $U$ , i.e. any element of  $\mathbf{Con}_U$ , is equivalent to the initial morphism  $F$  we just defined.

## 1.7 Problem Definition

In this section we will introduce a formal definition of our problem, proposed at the beginning of the [thesis](#), and reformulate it to an equivalent problem easier to solve with artificial intelligence.

Our problem might be formalized in the following manner: given a theory  $U$  and a derived rule of  $U$  of the form

$$\frac{x_1 : A_1, \dots, x_n : A_n}{A_{\text{type}}}$$

we want to find an expression  $t$  of  $U$  such that the rule

$$\frac{x_1 : A_1, \dots, x_n : A_n}{t : A}$$

is a derived rule of  $U$ .

This problem is undecidable, in general, as it follows from the undecidability of the inhabitation in a *GAT*. This is why we won't try to find an algorithm that either finds a term or says that the type is empty. We aim to find an algorithm that in the case that the type is inhabited, it will find a term in a finite number of steps. In the case that the type is empty, it does not have to give an output.

We can consider this problem from the point of view of the context category  $C(U)$ , where by the completeness theorem 1.15 it is rephrased as: given a context  $\langle x_1 : A_1, \dots, x_n : A_n \rangle$  in  $C(U)$ , we want to find a section of  $\langle x_1 : A_1, \dots, x_n : A_n, x : A \rangle$  in  $C(U)$ .

By naming  $\Gamma \equiv \langle x_1 : A_1, \dots, x_n : A_n \rangle$  we can restrict our search to the *relativization* of  $C(U)$  to  $[\Gamma]$ , that we will note as  $\mathcal{C}_\Gamma$ . We will note contexts of the form  $\Gamma, y_1 : B_1, \dots, y_m : B_m$  with the subindex  $\Gamma$  in the following way

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$$

to serve as a shorthand notation of objects of  $\mathcal{C}_\Gamma$ . The same notation will be used when talking about morphisms in  $\mathcal{C}_\Gamma$ , by noting  $\langle x_1, \dots, x_n, t_{n+1}, \dots, t_{n+m} \rangle$  as  $\langle t_{n+1}, \dots, t_{n+m} \rangle_\Gamma$ .

## 1.8 Expansion Algorithm

In this section we will define the *expansion algorithm* that will be responsible of finding  $\langle t \rangle_\Gamma$  in  $\mathcal{C}_\Gamma$ . The complexity of this algorithm is really high, and it will serve as a baseline to be optimized by a graph neural network, we will explain this in the last chapter.

First, consider  $G$  a finite subcategory of  $\mathcal{C}_A$  such that it contains  $\Gamma, \langle x : A \rangle_\Gamma$  and its projection. The process will consist in the construction of a sequence  $\{G_i\}_i$  of finite subcategories of  $\mathcal{C}_\Gamma$  such that  $G_0 = G$ , and that  $G_{n+1}$  is computed by the process described below from  $G_n$ , and where  $G_n \subseteq G_{n+1}$  for all  $n \geq 0$ . We will say that an object or morphism of  $\mathcal{C}_A$  is **constructible by expansion** if there exists a  $n \geq 0$  such that the given object or morphism is contained in  $G_n$ . The process of constructing  $G_{n+1}$  from  $G_n$  consists on applying the following rules to each context  $\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$  of  $G_n$ :

E1 For each sort symbol  $S$  with a well-formed introductory rule

$$\frac{z_1 : C_1, \dots, z_n : C_w}{A(z_1, \dots, z_w) \text{type}}$$

such that  $G_n$  contains sections

$$\begin{aligned} \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_1 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1 \rangle_\Gamma \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_2 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_2 : C_2[z_1 | t_1] \rangle_\Gamma \\ &\vdots \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_w \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_w : C_w[z_1 | t_1, \dots, z_{w-1} | t_{w-1}] \rangle_\Gamma \end{aligned}$$

then we add the context  $\langle y_1 : B_1, \dots, y_m : B_m, y : A(t_1, \dots, t_w) \rangle_\Gamma$  and its projection to  $G_{n+1}$

E2 add the morphisms

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, y_i \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, y : B_i \rangle_\Gamma$$

and their corresponding projections for each  $0 \leq i \leq n$ .

E3 for every operator symbol  $F$  with a well-formed introductory rule

$$\frac{z_1 : C_1, \dots, z_w : C_w}{F(z_1, \dots, z_w) : C}$$

if the following sections exist in  $G_n$

$$\begin{aligned} \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_1 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1 \rangle_\Gamma \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_2 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_2 : C_2[z_1 | t_1] \rangle_\Gamma \\ &\vdots \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma &\xrightarrow{\langle y_1, \dots, y_m, t_w \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_w : C_w[z_1 | t_1, \dots, z_{w-1} | t_{w-1}] \rangle_\Gamma \end{aligned}$$

then we add the section

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, F(t_1, \dots, t_w) \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z : C[z_1 | t_1, \dots, z_w | t_w] \rangle_\Gamma$$

E4 for every morphism

$$\langle t_1, \dots, t_{m-1} \rangle_\Gamma : \langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma \rightarrow \langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma$$

we construct the corresponding pullback

$$\begin{array}{ccc} \langle z_1 : C_1, \dots, z_w : C_w, y_m : B_m[t_1 | y_1, \dots, t_{m-1} | y_{m-1}] \rangle_\Gamma & \xrightarrow{\langle t_1, \dots, t_{m-1}, y_m \rangle_\Gamma} & \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \\ \downarrow & & \downarrow \\ \langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma & \xrightarrow{\langle t_1, \dots, t_{m-1} \rangle_\Gamma} & \langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma \end{array}$$

in  $G_{n+1}$ .

**Theorem 1.16.** *Every object and every morphism of  $\mathcal{C}_A$  is constructible by expansion.*

*Proof.* We will prove this result by structural induction over the underlying derived rules corresponding to such object (context) or morphism (realization).

Consider a context  $\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$  of  $U$ , which is the representative of an object in  $C(U)$ . This means, by the definition of context that its underlying rule

$$\frac{\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}}{B_m \text{ type}}$$

is a derived rule of  $U$  and  $y_m$  is a variable different to  $x_1, \dots, x_n, y_1, \dots, y_{m-1}$ . Hence, the premise  $\langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma$  is a context, and by induction it is

constructible by expansion, i.e. there exists  $n_0 \geq 0$  such that  $\langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma$  is an object  $G_{n_0}$ . As (1.8) is a derived rule, it must have been derived by the only derivation rule that has a  $T$ -rule as a conclusion: CF2(a). Hence, there exists a sort symbol  $S$  with a well formed introductory rule  $\frac{z_1 : C_1, \dots, z_w : C_w}{S(z_1, \dots, z_w) \text{type}}$  and derived rules

$$\begin{array}{c} \frac{\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}}{t_1 : C_1}, \\ \frac{\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}}{t_2 : C_2[t_1|z_1]}, \\ \vdots \\ \frac{\Gamma y_1 : B_1, \dots, y_{m-1} : B_{m-1}}{t_w : C_w[t_1|z_1, \dots, t_{w-1}|z_{w-1}]} \end{array}$$

By induction hypothesis their corresponding sections

$$\langle y_1, \dots, y_{m-1}, t_j \rangle_\Gamma : \langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma \rightarrow \langle y_1 : B_1, \dots, y_{m-1} : B_{m-1}, z_j : C_j[t_1|z_1, \dots, t_{j-1}|z_{j-1}] \rangle_\Gamma$$

are constructible by expansion, so for each  $j$  there exists some  $n_j$  such that the section exists in  $G_{n_j}$ .

Take  $n = \max\{n_0, n_1, \dots, n_w\}$ . Then, all of the objects, and sections mentioned belong to  $G_n$  so by the expansion rule E1,

$$\langle y_1 : B_1, \dots, y_{m-1} : B_{m-1} \rangle_\Gamma \longleftarrow \langle y_1 : B_1, \dots, y_{m-1} : B_{m-1}, y_m : S(t_1, \dots, t_w) \rangle_\Gamma$$

is included in  $G_{n+1}$ . And by the equivalence of intended equivalence of denotation

$$\langle y_1 : B_1, \dots, y_{m-1} : B_{m-1}, y_m : S(t_1, \dots, t_w) \rangle_A \equiv \langle y_1 : B_1, \dots, y_m : B_m \rangle_A$$

so  $\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$  is constructible by expansion.

Now consider a morphism

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle t_1, \dots, t_w \rangle_\Gamma} \langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma$$

a representative of a morphism in  $\mathcal{C}_A$ . By induction hypothesis, we know that both  $\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$  and  $\langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma$  are constructible by extension.

Moreover, as  $\langle t_1, \dots, t_w \rangle_\Gamma$  is an operator, for each  $1 \leq j \leq w$ , the rule

$$\frac{\Gamma y_1 : B_1, \dots, y_m : B_m}{t_j : C_j[z_1 \mid t_1, \dots, z_{j-1} \mid t_{j-1}]}$$

is a derived rule of  $U$ . Now we will see that the following sections are constructible by expansion:

$$\begin{aligned}
&\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_1 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1 \rangle_\Gamma \\
&\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_2 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_2 : C_2[t_1|z_1] \rangle_\Gamma \\
&\quad \vdots \\
&\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_w \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_w : C_w[t_1|z_1, \dots, t_{w-1}|z_{w-1}] \rangle_\Gamma
\end{aligned}$$

To do this, we will use the way any rule (1.8) are derived. This can be done by one of the following rules: T1, CF1, CF2(b). Hence,

- if derived by T1, we have that both

$$\frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{A = C_j[z_1 \mid t_1, \dots, z_{j-1}]} \quad \text{and} \quad \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{t_j : A}$$

are derived rules of  $U$ . So by the first rule, due to the equivalence relation of intended equality of denotation, we know that

$$\begin{aligned}
\langle y_1 : B_1, \dots, y_m : B_m, z_j : A \rangle_\Gamma &\equiv \\
&\langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[t_1|z_1, \dots, t_{j-1}|z_{j-1}] \rangle_\Gamma
\end{aligned}$$

hence, they are the same object in  $\mathcal{C}_\Gamma$ . And by the second one, applying induction, we have that

$$\text{lang} \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_j \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : A \rangle_\Gamma$$

is constructible by expansion, so as  $\langle y_1 : B_1, \dots, y_m : B_m, z_j : A \rangle_\Gamma$  and  $\langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[t_1|z_1, \dots, t_{j-1}|z_{j-1}] \rangle_\Gamma$  are the same object in  $\mathcal{C}_A$ , then

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_j \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[t_1|z_1, \dots, t_{j-1}|z_{j-1}] \rangle_\Gamma$$

is constructible by expansion.

- if derived by CF1, we have that

$$\frac{\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}}{B_m \text{ type}}$$

is a derived rule of  $U$ , where  $y_m$  is a variable distinct from all  $x_1, \dots, x_n, y_1, \dots, y_{m-1}$ , and that there exists an  $i$ ,  $1 \leq i \leq m$ , such that

$$\frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{t_j : C[z_1 \mid t_1, \dots, z_{j-1} \mid t_{j-1}]} \equiv \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{y_i : B_i}$$



Then, by induction hypothesis, the context  $\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma$  is constructible by expansion. By applying the expansion rule E2 and an arguments of equivalent to the one above, we get that

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_j \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[z_1 \mid t_1, \dots, z_{j-1} \mid t_{j-1}] \rangle_\Gamma$$

is constructible by expansion.

- if derived by CF2(b), there exists an operator symbol  $F$  with a well-formed introductory rule

$$\frac{v_1 : D_1, \dots, v_k : D_k}{F(v_1, \dots, v_k) : D}$$

and derived rules

$$\begin{array}{c} \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{s_1 : D_1} \\ \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{s_2 : D_2[v_1 | s_1]} \\ \vdots \\ \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{s_v : D_k[v_1 | s_1, \dots, v_{k-1} | s_{k-1}]} \end{array}$$

such that

$$\begin{array}{c} \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{D[s_1 | c_1, \dots, s_v | c_v] = C_j[t_1 | z_1, \dots, t_{j-1} | z_{j-1}]} \\ \frac{\Gamma, y_1 : B_1, \dots, y_m : B_m}{t_j = F(s_1, \dots, s_v) : C_j[t_1 | z_1, \dots, t_{j-1} | z_{j-1}]} \end{array}$$

are derived rules of  $U$ . By the equivalence relation of identity of denotation, we have to prove that the morphism

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, F(s_1, \dots, s_k) \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : D[s_1 | c_1, \dots, s_k | v_k] \rangle_\Gamma$$

is constructible by expansion. By induction we have that the following morphisms are constructible by expansion:

$$\begin{array}{c} \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, s_1 \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, v_1 : D_1 \rangle_\Gamma \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_A \xrightarrow{\langle y_1, \dots, y_m, s_2 \rangle_A} \langle y_1 : B_1, \dots, y_m : B_m, v_2 : D_2[s_1 | v_1] \rangle_\Gamma \\ \vdots \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, s_v \rangle_A} \langle y_1 : B_1, \dots, y_m : B_m, c_v : D_k[s_1 | v_1, \dots, s_{k-1} | v_{k-1}] \rangle_A \end{array}$$

Hence, by applying the expansion rule E3, we have that

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, F(s_1, \dots, s_k) \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : D_j[s_1|v_1, \dots, s_k|c_k] \rangle_\Gamma$$

is constructible by expansion as we wanted.

Now we will show that the morphism

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_1, \dots, t_w \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma$$

is constructible by expansion. To do this we will prove that given a  $j$ ,  $2 < j \leq n$ , if

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_1, \dots, t_{j-1} \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_{j-1} : C_{j-1} \rangle_\Gamma$$

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_j \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[z_1 \mid \dots z_{j-1} \mid t_{j-1}] \rangle_\Gamma$$

are constructible by expansion so is

$$\langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma \xrightarrow{\langle y_1, \dots, y_m, t_1, \dots, t_j \rangle_\Gamma} \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_j : C_j \rangle_\Gamma$$

By expansion rule E4, the following diagram is constructible by expansion:

$$\begin{array}{ccc} \langle y_1 : B_1, \dots, y_m : B_m, z_j : C_j[t_1|z_1, \dots, t_j|z_j] \rangle_\Gamma & \xrightarrow{\langle y_1, \dots, y_m, t_1, \dots, t_{j-1}, t_j \rangle_\Gamma} & \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_j : C_j \rangle_\Gamma \\ \downarrow & & \downarrow \\ \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma & \xrightarrow{\langle y_1, \dots, y_m, t_1, \dots, t_{j-1} \rangle_\Gamma} & \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_{j-1} : C_{j-1} \rangle_\Gamma \end{array}$$

and by composition we get that

$$\langle y_1, \dots, y_m, t_1, \dots, t_{j-1}, z_j \rangle_\Gamma \circ \langle y_1, \dots, y_m, t_j \rangle_\Gamma = \langle y_1, \dots, y_m, t_1, \dots, t_j \rangle_\Gamma$$

is constructible by expansion. It remains to see that the morphism

$$\langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma \xrightarrow{\langle z_1, \dots, z_w \rangle_\Gamma} \langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma$$

is constructible by expansion. That comes directly from the fact that  $\langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma$  is a context, and the repeated application of the rule E4 gives us the following di-

agram:

$$\begin{array}{ccc}
 \langle y_1 : B_1, \dots, y_m : B_m z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma & \xrightarrow{\langle z_1, \dots, z_w \rangle_\Gamma} & \langle z_1 : C_1, \dots, z_w : C_w \rangle_\Gamma \\
 \downarrow & & \downarrow \\
 \langle y_1 : B_1, \dots, y_m : B_m z_1 : C_1, \dots, z_{w-1} : C_{w-1} \rangle_\Gamma & \xrightarrow{\langle z_1, \dots, z_{w-1} \rangle_\Gamma} & \langle z_1 : C_1, \dots, z_{w-1} : C_{w-1} \rangle_\Gamma \\
 \downarrow & & \downarrow \\
 \vdots & & \vdots \\
 \downarrow & & \downarrow \\
 \langle y_1 : B_1, \dots, y_m : B_m, z_1 : C_1 \rangle_\Gamma & \xrightarrow{\langle z_1 \rangle_\Gamma} & \langle z_1 : C_1 \rangle_\Gamma \\
 \downarrow & & \downarrow \\
 \langle y_1 : B_1, \dots, y_m : B_m \rangle_\Gamma & \xrightarrow{\langle \rangle_\Gamma} & \langle \rangle_\Gamma
 \end{array}$$

□



## Chapter 2

# The Calculus of Constructions

In Chapter 1, we established a pipeline for autonomous theorem proving: take a theory, translate its syntax into a Contextual Category, and reframe proof-finding as the geometric problem of finding a section in a graph.

In this chapter, we scale up the complexity to the Calculus of Constructions (CoC). Introduced by Coquand and Huet [4], CoC is a powerful high-order typed lambda calculus. It serves as the foundation for the Calculus of Inductive Constructions, the type theory powering modern proof assistants like Rocq (formerly Coq).

The primary additions in CoC, compared to Generalized Algebraic Theories (GATs), are:

1. **Product Types ( $\Pi$ -types):** These are first-class citizens, meaning dependent function types exist at the base level rather than just in the theory's signature.
2. **The universe of Propositions (Prop):** A type universe whose terms are themselves types, allowing for higher-order logic and universal quantification over types.

We will define syntax of CoC, map it to a richer categorical structure called Doctrine of Constructions, and extend our Expansion Algorithm to navigate this new, highly complex search space.

### 2.1 Syntactic Formulation

In this section, we will formally define the syntax of Calculus of Constructions. We will use the approach of Thomas Streicher in [6] as the original definition in [4] is characterized by its syntax overloading, making it much harder to follow. They just use  $[x : A]B$  to present product types, functional abstraction and universal

quantification. Also, properties are presented as terms of type *Prop* and as types themselves. To solve the latter, we will be using a type constructor *Proof* that given a proposition  $p : \text{Prop}$  it will give  $\text{Proof}(p)$  the type of proofs of proposition  $p$ .

First, we will define the class of **pre-well-formed type expressions**, ranged over by capital letters, and the class of **pre-well-formed object expressions**, ranged over by lower case letters, by mutual recursion in a BMF-like way:

$$\begin{aligned}
 E ::= & \quad (\Pi x : E) E && \text{product of types} \\
 & \quad | \text{Prop} && \text{type of propositions} \\
 & \quad | \text{Proof}(t) && \text{type of proofs of proposition } t \\
 t ::= & \quad x && \text{variable} \\
 & \quad | (\lambda x : E) t && \text{functional abstraction} \\
 & \quad | \text{App}([x : E]E, t, t) && \text{function application} \\
 & \quad | (\forall x : E) t && \text{universal quantification}
 \end{aligned}$$

We say a **pre-context** is a syntactical expression of the form  $x_1 : A_1, \dots, x_n : A_n$ , where  $x_1, \dots, x_n$  are pairwise syntactically different variables. Contexts are ranged over by capital greek letters. We define  $\text{Var}(\Gamma)$  as the set of variables declared in  $\Gamma$ . We will not distinguish expressions which are equivalent up to renaming of variables ( $\alpha$ -conversion), to do this we will be using *deBruijn indices*<sup>1</sup>. We define a **pre-judgment** to be a syntactic expression of one of the following forms:

$$\begin{aligned}
 A \text{ type} & \quad A \text{ is a type} \\
 A = B & \quad A \text{ and } B \text{ are equal types} \\
 t : A & \quad t \text{ is an object of type } A \\
 t = s : A & \quad t \text{ and } s \text{ are equal objects of type } A
 \end{aligned}$$

we will use the letter  $\mathcal{J}$  to refer to pre-judgments. We define a **pre-sequent** as a syntactic expression of the following form

$$\begin{aligned}
 \Gamma \text{ ok} & \quad \Gamma \text{ is a context} \\
 \Gamma = \Delta & \quad \Gamma \text{ and } \Delta \text{ are equal contexts} \\
 \Gamma \vdash \mathcal{J} & \quad \text{the judgement } \mathcal{J} \text{ holds w.r.t. } \Gamma
 \end{aligned}$$

Finally we define the set of **sequents**, i.e. valid pre-sequents, to be generated by the following set of rules. Sequents are ranged over by capital italic letters. This set of rules is more extent that the original in order to make explicit rules for handling contexts and to add more rules stating equality of types and objects.

<sup>1</sup>*deBruijn indices* are a method to refer to declarations not by name (variables) but by numbers which tell how far one has to go back into the current context of an expression to find the declaration of the thing it refers to.

When presenting the axiomatization of Calculus of Constructions we shall use the following notation:

$$\frac{S_1}{S_2} \text{ abbreviates } \frac{S_1}{S_2} \text{ and } \frac{S_2}{S_1}$$

Context Formation Rules:

$$\text{EMPTY} \frac{}{\text{ok}}$$

$$\text{CONT-INTRO} \frac{\Gamma \vdash A_{\text{type}}}{\Gamma, x : A_{\text{ok}}}, \text{ if } x \notin \text{Var}(\Gamma)$$

$$\text{CONT-THIN} \frac{\Gamma, \Delta_{\text{ok}} \quad \Gamma \vdash A_{\text{type}}}{\Gamma, x : A, \Delta_{\text{ok}}}, \text{ if } x \notin \text{Var}(\Gamma) \cup \text{Var}(\Delta)$$

$$\text{CONT-SUB} \frac{\Gamma, x : A, \Delta_{\text{ok}} \quad \Gamma \vdash t : A}{\Gamma, \Delta[x \mid t]_{\text{ok}}}$$

Context Equality Rules

$$\text{CREFL} \frac{\Gamma = \Gamma}{\Gamma_{\text{ok}}}$$

$$\text{CSYM} \frac{\Gamma = \Delta}{\Delta = \Gamma}$$

$$\text{CTRANS} \frac{\Gamma = \Delta \quad \Delta = \Theta}{\Gamma = \Theta}$$

$$\text{CEQU} \frac{\Gamma, x : A, \Delta_{\text{ok}} \quad \Gamma \vdash A = B}{\Gamma, x : A, \Delta = \Gamma, x : B, \Delta}$$

$$\text{CREFLECT1} \frac{\Gamma, x : A = \Delta, x : B}{\Gamma = \Delta}$$

$$\text{CREFLECT2} \frac{\Gamma, x : A = \Delta, x : B}{\Gamma \vdash A = B_1}$$

Structural Rules for Judgement Formation

$$\text{ASS} \frac{\Gamma, x : A, \Delta_{\text{ok}}}{\Gamma, x : A, \Delta \vdash x : A}$$

$$\text{REFLECTION1} \frac{\Gamma \vdash \mathcal{J}}{\Gamma_{\text{ok}}}$$

$$\text{REFLECTION2} \frac{\Gamma \vdash t : A}{\Gamma \vdash A_{\text{type}}}$$

$$\text{THIN} \frac{\Gamma, \Delta \vdash \mathcal{J} \quad \Gamma \vdash A_{\text{type}}}{\Gamma, x : A, \Delta \vdash \mathcal{J}}, \text{ if } x \notin \text{Var}(\Gamma) \cup \text{Var}(\Delta)$$

$$\text{SUB} \frac{\Gamma, x : A, \Delta \vdash \mathcal{J} \quad \Gamma \vdash t : A}{\Gamma, \Delta[x \mid t] \vdash \mathcal{J}[x \mid t]}$$

Equality Rules:

$$\text{EQU} \frac{\Gamma = \Delta \quad \Gamma \vdash \mathcal{J}}{\Delta \vdash \mathcal{J}}$$

$$\text{REFL-TYP} \frac{\Gamma \vdash A = A}{\Gamma \vdash A \text{type}}$$

$$\text{REFL-OBJ} \frac{\Gamma \vdash t = t : A}{\Gamma \vdash t : A}$$

$$\text{SYMM-TYP} \frac{\Gamma \vdash A = B}{\Gamma \vdash B = A}$$

$$\text{SYMM-OBJ} \frac{\Gamma \vdash t = s : A}{\Gamma \vdash s = t : A}$$

$$\text{TRANS-TYP} \frac{\Gamma \vdash A = B \quad \Gamma \vdash B = C}{\Gamma \vdash A = C}$$

$$\text{TRANS-OBJ} \frac{\Gamma \vdash t_1 = t_2 : A \quad \Gamma \vdash t_2 = t_3 : A}{\Gamma \vdash t_1 = t_3 : A}$$

$$\text{REPL1} \frac{\Gamma \vdash t = s : A \quad \Gamma, x : A, \Delta \text{ok}}{\Gamma, \Delta[x \mid t] = \Gamma, \Delta[x \mid s]}$$

$$\text{REPL2} \frac{\Gamma \vdash t = s : A \quad \Gamma, x : A, \Gamma \vdash B \text{type}}{\Gamma, \Delta[x \mid t] \vdash B[x \mid t] = B[x \mid s]}$$

$$\text{REPL3} \frac{\Gamma \vdash t_1 = t_2 : A \quad \Gamma, x : A, \Delta \vdash s : B}{\Gamma, \Delta[x \mid t_1] \vdash s[x \mid t_1] = s[x \mid t_2] : B[x \mid t_1]}$$

$$\text{CONV1} \frac{\Gamma \vdash A = B \quad \Gamma \vdash t : A}{\Gamma \vdash t : B}$$

$$\text{CONV2} \frac{\Gamma \vdash A = B \quad \Gamma \vdash t = s : A}{\Gamma \vdash t = s : B}$$

Rules for Products of types

$$\Pi\text{-FORM} \frac{\Gamma, x : A \vdash B \text{type}}{\Gamma \vdash (\Pi x : A) B \text{type}}$$

$$\Pi\text{-EQU} \frac{\Gamma \vdash A_1 = A_2 \quad \Gamma, x : A_1 \vdash B_1 = B_2}{\Gamma \vdash (\Pi x : A_1) B_1 = (\Pi x : A_2) B_2}$$

$$\Pi\text{-INTRO} \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash (\lambda x : A) t : (\Pi x : A) B}$$



$$\xi\text{-RULE} \frac{\Gamma, x: A_1 \vdash t_1 = t_2 : B \quad \Gamma \vdash A_1 = A_2}{\Gamma \vdash (\lambda x: A_1) t_1 = (\lambda x: A_2) t_2 : (\prod x: A_1) B}$$

$$\prod\text{-ELIM} \frac{\Gamma \vdash t: (\prod x: A) B \quad \Gamma \vdash s: A \quad \Gamma, x: A \vdash B \text{type}}{\Gamma \vdash \text{App}([x: A] B, t, s): B[x \mid s]}$$

$$\prod\text{-ELIM-EQU} \frac{\Gamma \vdash A_1 = A_2 \quad \Gamma \vdash t_1 = t_2 : (\prod x: A_1) B_1 \quad \Gamma, x: A_1 \vdash B_1 = B_2 \quad \Gamma \vdash s_1 = s_2 : A_1}{\Gamma \vdash \text{App}([x: A_1] B_1, t_1, s_1) = \text{App}([x: A_2] B_2, t_2, s_2) : B_1[x \mid s_1]}$$

$$\beta\text{-RULE} \frac{\Gamma, x: A \vdash B \text{type} \quad \Gamma, x: A \vdash t: B \quad \Gamma \vdash s: A}{\Gamma \vdash \text{App}([x: A] B, (\lambda x: A) t, s) = t[x \mid s] : B[x \mid s]}$$

$$\eta\text{-RULE} \frac{\Gamma, x: A \vdash B \text{type} \quad \Gamma \vdash t: (\prod x: A) B}{\Gamma \vdash (\lambda x: A) \text{App}([x: A] B, t, x) = t : (\prod x: A) B}$$

Rules for Propositions

$$\text{PROP} \frac{\Gamma \text{ok}}{\Gamma \vdash \text{Proptype}}$$

$$\text{PROP-TYPE} \frac{\Gamma \vdash p: \text{Prop}}{\Gamma \vdash \text{Proof}(p) \text{type}}$$

$$\text{PROP-EQU} \frac{\Gamma \vdash p_1 = p_2 : \text{Prop}}{\Gamma \vdash \text{Proof}(p_1) = \text{Proof}(p_2)}$$

$$\forall\text{-INTRO} \frac{\Gamma, x: A \vdash p: \text{Prop}}{\Gamma \vdash (\forall x: A) p: \text{Prop}}$$

$$\forall\text{-EQU} \frac{\Gamma, x: A_1 \vdash p_1 = p_2 : \text{Prop} \quad \Gamma \vdash A_1 = A_2}{\Gamma \vdash (\forall x: A_1) p_1 = (\forall x: A_2) p_2 : \text{Prop}}$$

$$\forall\text{-ELIM} \frac{\Gamma, x: A \vdash p \in \text{Prop}}{\Gamma \vdash \text{Proof}((\forall x: A) p) = (\prod x: A) \text{Proof}(p)}$$

## 2.2 Doctrines of Constructions

We will leverage the concept of context categories introduced in the earlier chapter to serve as a basic structure for handling dependent types, and add more conditions so that it can handle products of families of types, universal quantification and the type of properties, in order to develop a categorical structure that captures the interpretation of the Calculus of Constructions.

Although products might be added directly in the GAT structure, we will enforce them to the base level as they are needed to the definition of universal quantification.

**Definition 2.1.** A *contextual category with products of families of types* is a category  $\mathcal{C}$  together with two mappings  $\prod$  and  $eval$  defined over the collection of objects  $B$  of  $\mathcal{C}$  with  $level(B) \geq 2$ , such that:

1. whenever  $1 \triangleleft A \triangleleft B$ , then  $1 \triangleleft \prod(B)$  and  $eval_B: p(A)^* \prod(B) \rightarrow B$  is a morphism over  $A$

$$\begin{array}{ccc} p(A)^* \prod(B) & \xrightarrow{eval_B} & B \\ & \searrow p(p(A)^* \prod(B)) & \swarrow p(B) \\ & A & \end{array}$$

such that for all  $s \in Sections(B)$  there is a unique section  $t \in Sections(\prod(B))$  with  $eval_B \circ p(A)^* t = s$ . This unique section  $t$  is called  $curry(s)$ .

$$\begin{array}{ccccc} & A & \xrightarrow{\quad} & 1 & \\ & \downarrow p(A)^* t & & \downarrow t & \\ id_A \swarrow & p(A)^* \prod(B) & \xrightarrow{q(p(A), \prod(B))} & \prod(B) & \searrow id_1 \\ & \downarrow eval_B & & \downarrow & \\ & B & & & \\ & \downarrow & & & \\ & A & \xrightarrow{\quad} & 1 & \end{array}$$

**Observation 2.2.** as any section  $t$  of  $\prod(B)$  gives a section  $eval_B \circ p(A)^* t$  of  $B$ , hence we have a canonical 1-1-correspondence of  $Sections(B)$  and  $Sections(\prod(B))$ .

2. and for any object  $J$  and any morphism  $f: J \rightarrow 1$  the following conditions are satisfied

$$f^*(\prod(B)) = \prod(f^* B) \quad \text{and} \quad f^*(eval_B) = eval_{f^* B}$$

It follows from the definition that substitution preserves functional abstraction or currying:

**Lemma 2.3.** Let  $\mathcal{C}$  be a contextual category with products of families of types. Let  $1 \triangleleft A \triangleleft B$  be objects and  $f: J \rightarrow 1$  a morphism in  $\mathcal{C}$ . If  $t$  is a section of  $B$ , then

$$f^*(curry(B)) = curry(f^* B)$$

Notice that in any contextual category with products of families of types, the following equations hold:

$$\text{eval}_B \circ p(A)^* \text{curry}(s) = s \quad (\beta)$$

$$\text{curry}(\text{eval}_B \circ p(A)^* t) = t \quad (\eta)$$

The first equation corresponds to the  $\beta$ -rule of typed  $\lambda$ -calculi and the second equation corresponds to the  $\eta$ -rule of typed  $\lambda$ -calculi. Contextual Categories with products correspond to  $\lambda$ -calculus with dependent types.

**Definition 2.4.** Let  $\mathcal{C}, \mathcal{C}'$  be two contextual categories with products of families of types, we a contextual functor  $F: \mathcal{C} \rightarrow \mathcal{C}'$  **preserves products of families of types** if for  $1 \triangleleft A \triangleleft B$  in  $\mathcal{C}$ ,

$$F \prod(B) = \prod(FB) \quad \text{and} \quad F \text{eval}_B = \text{eval}_{FB}$$

NOTE: this definition is better for actually proving relativization on the section on Problem Redefinition An alternative definition of contextual categories with products of families of types is given by Vladimir Voevodsky in this [INSERT REF](#)

**Definition 2.5.** Let  $\mathcal{C}$  be a category. Let  $g: Z \rightarrow Y, f: Y \rightarrow X$  be a pair of morphisms such that for any  $U \rightarrow X$  there exists a pullback  $U \times_X Y$ . A pair

$$(w: W \rightarrow X, h: W \times_X Y \rightarrow Z)$$

such that  $g \circ h = \text{pr}$  is called a universal pair for  $(f, g)$  if for any  $U \rightarrow X$  the map

$$\begin{aligned} \text{Hom}_X(U, W) &\longrightarrow \text{Hom}_Y(U \times_X Y, Z) \\ u &\longmapsto h \circ (u \times \text{id}_Y) \end{aligned}$$

is a bijection.

**Definition 2.6.** a contextual category with products of families of types (or a  $\prod$ -contextual category in the reference) consist of a contextual category  $\mathcal{C}$  together with:

1. for each object  $Y$  of  $\mathcal{C}$ , with  $\text{level}(Y) \geq 2$ , an object  $\prod(Y)$  of  $\mathcal{C}$  such that  $\text{father}(\prod(Y)) = \text{father}^2(Y)$ .
2. for each object  $Y$  of  $\mathcal{C}$ , with  $\text{level}(Y) \geq 2$ , a morphism

$$\text{eval}_Y: p(\text{father}(Y))^* \prod(Y) \rightarrow Y$$

over  $\text{father}(Y)$  such that:

(a) for any  $f: Z \rightarrow \text{father}^2(Y)$

$$f^* \prod(Y) = \prod(f^*Y) \quad \text{and} \quad f^* \text{eval}_Y = \text{eval}_{f^*Y}$$

(b)  $(\prod(Y), \text{eval}_Y)$  is a universal pair for  $(p(Y), p(\text{father}(Y)))$ .

Now we will add the type of propositions. To do it, we shall find objects in our category whose sections corresponds, i.e. its terms, correspond to objects.

**Definition 2.7.** Let  $\mathcal{C}$  be a context category, an **enumeration of types of  $\mathcal{C}$**  is an object  $T$  of  $\mathcal{C}$  with  $\text{level}(T) = 2$ .

An enumeration of types  $T$  of  $\mathcal{C}$  is called a **collection of types of  $\mathcal{C}$**  if, and only if, for all objects  $A$  and morphisms  $f, g: A \rightarrow \text{father}(T)$

$$f^*T = g^*T \implies f = g$$

it is the object  $\text{father}(T)$  that we are interested in.

And finally, by wrapping everything up and adding universal quantification we have the desired structure:

**Definition 2.8.** a **doctrine of constructions** is a contextual category with products of families of types together with a collection of types  $T$  such that for all  $A, B \in \text{Ob}\mathcal{C}$  with  $A \triangleleft B$  and  $f: B \rightarrow \text{Prop}$ , where  $\text{Prop} = \text{father}T$ , there exists a necessarily unique morphism  $g: A \rightarrow \text{Prop}$  also denoted by  $\forall(f)$  such that

$$g^*T = \forall(f)^*T = \prod(f^*T)$$

**Definition 2.9.** Let  $\mathcal{C}, \mathcal{C}'$  be two doctrines of construction, a **functor of doctrines of constructions** is a contextual functor  $F: \mathcal{C} \rightarrow \mathcal{C}'$  that preserves products of families of types such that if  $T$  and  $T'$  are the selected collections of types of  $\mathcal{C}$  and  $\mathcal{C}'$  respectively,  $FT = T'$ , and for any  $A \triangleleft B$  in  $\mathcal{C}$ , and  $f: B \rightarrow \text{Prop}$ ,  $F(\forall(f)) = \forall(F(f))$ .

We will consider the category **DoC** of doctrines of constructions and functors of doctrines of constructions.

Now we will see that doctrines of constructions correspond to models of Calculus of Constructions.

## 2.3 Interpretation of Calculus of Constructions

In this section we will define the interpretation of Calculus of Constructions in arbitrary doctrines of constructions.

To do this, fixed one doctrine of constructions, an **interpretation function**  $\mathcal{M}$  should be a mapping such that:

1. for every pre-context  $\Gamma$  of length  $n$ , if  $\mathcal{M}[\Gamma]$  is defined, then  $\mathcal{M}[\Gamma]$  is an object of level  $n$ ;
2. for every pre-context of length  $n$  and type expression  $A$ , if  $\mathcal{M}[\Gamma; A]$  is defined then  $\mathcal{M}[\Gamma; A]$  is an object of level  $n + 1$  and  $\mathcal{M}[\Gamma] \triangleleft \mathcal{M}[\Gamma; A]$ ;
3. for every pre-context  $\Gamma$  of length  $n$  and object expression  $t$ , if  $\mathcal{M}[\Gamma; t]$  is defined then it is a morphism  $s : \mathcal{M}[\Gamma] \rightarrow A$  where  $A$  is an object of level  $n + 1$  such that

$$\mathcal{M}[\Gamma] \triangleleft A \quad \text{and} \quad p(A) \circ s = \text{id}_{\mathcal{M}[\Gamma]}$$

We want to build this function by induction, we need an auxiliary definition:

**Definition 2.10.** *by structural induction of expressions we define a function **depth** mapping pre-well-formed expressions and pre-contexts to  $\omega$ :*

$$\text{depth}[x] = 1$$

$$\text{depth}[\text{Prop}] = 1$$

$$\text{depth}[\text{Proof}(A)] = \text{depth}[A] + 1$$

$$\text{depth}[(\prod x : A)B] = \text{depth}[A] + \text{depth}[B] + 1$$

$$\text{depth}[(\lambda x : A)t] = \text{depth}[A] + \text{depth}[t] + 1$$

$$\text{depth}[(\forall x : A)t] = \text{depth}[A] + \text{depth}[t] + 1$$

$$\text{depth}[\text{App}([x : A]B, t, s)] = \text{depth}[A] + \text{depth}[B] + \text{depth}[t] + \text{depth}[s] + 1$$

$$\text{For } \Gamma \equiv x_1 : A_1, \dots, x_n : A_n, \quad \text{depth}[\Gamma] = \sum_{i=1}^n \text{depth}[A_i]$$

We will define  $\mathcal{M}$  by induction on the level of its arguments where

$$\text{level}[\Gamma] = \text{depth}[\Gamma],$$

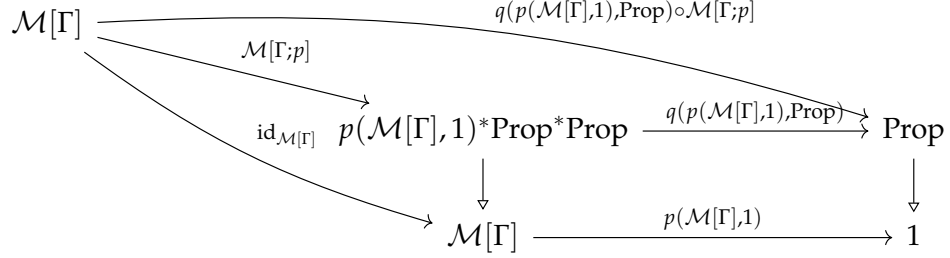
$$\text{level}[\Gamma; A] = \text{depth}[\Gamma] + \text{depth}[A],$$

$$\text{level}[\Gamma; t] = \text{depth}[\Gamma] + \text{depth}[t]$$

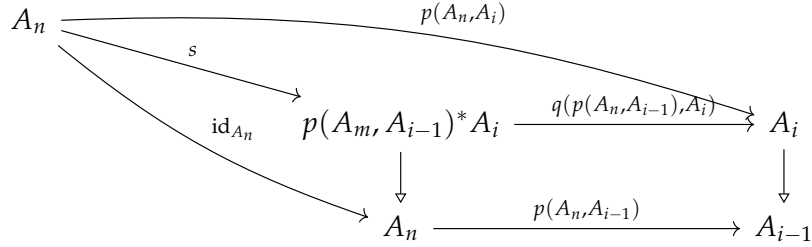
by the following clauses:

1.  $\mathcal{M}[] = 1$
2.  $\mathcal{M}[\Gamma, x : A] = \mathcal{M}[\Gamma; A]$
3.  $\mathcal{M}[\Gamma, (\prod x : A)B] = \prod (\mathcal{M}[\Gamma, x : A; B])$  if  $\mathcal{M}[\Gamma, x : A; B]$  is defined

4.  $\mathcal{M}[\Gamma, \text{Prop}] = p(\mathcal{M}, 1)^* \text{Prop}$  if  $\mathcal{M}[\Gamma]$  is defined
5.  $\mathcal{M}[\Gamma; \text{Proof}(p)] = \mathcal{M}[\Gamma; p]^* p(\mathcal{M}[\Gamma], 1)^* T$  if  $\mathcal{M}[\Gamma; p]$  is defined and a section of  $p(\mathcal{M}[\Gamma], 1)^* \text{Prop}$



6.  $\mathcal{M}[\Gamma, x] = s$  provided that  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n$  and  $x_i \equiv x$  and  $\mathcal{M}[\Gamma]$  is defined and  $\mathcal{M}[\Gamma] = A_n \triangleright \dots \triangleright A_1 \triangleright A_0 \triangleright 1$  and  $s$  as follows



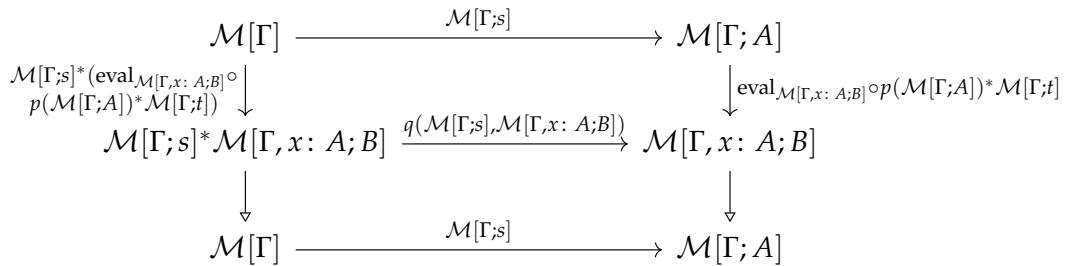
or in other terms  $\mathcal{M}[\Gamma; x_i] = p(A_n, A_i)^* \Delta(A_i)$  where  $\Delta(A_i)$  is the unique arrow  $m : A_i \rightarrow p(A_i)^* A_i$  such that

$$p(p(A_i)^* A_i) \circ m = q(p(A_i), A_i) \circ m = \text{id}_{A_i}$$

7.  $\mathcal{M}[\Gamma; (\lambda x : A) t] = \text{curry}(\mathcal{M}[\Gamma, x : A; t])$  if  $\mathcal{M}[\Gamma, x : A; t]$  is defined. By induction hypothesis we have that  $\mathcal{M}[\Gamma] \triangleleft \mathcal{M}[\Gamma; A] \triangleleft \text{cod}(\mathcal{M}[\Gamma, x : A; t])$  and  $s$  is a section of  $\mathcal{M}[\Gamma, x : A; t]$  and therefore we have:

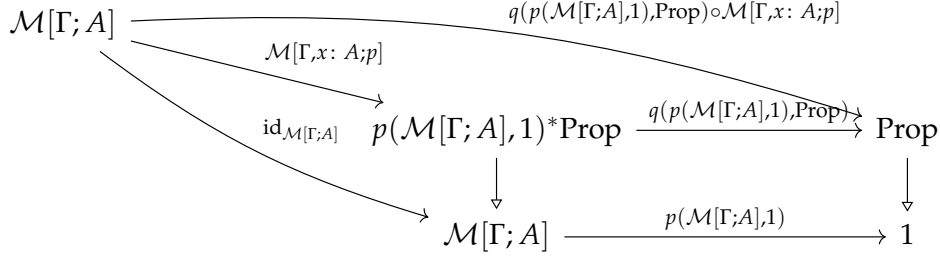
$$\text{eval}_{\text{cod}(\mathcal{M}[\Gamma, x : A; t])} \circ p(\mathcal{M}[\Gamma; A])^* \mathcal{M}[\Gamma; (\lambda x : A) t] = \mathcal{M}[\Gamma, x : A; t]$$

8.  $\mathcal{M}[\Gamma; \text{App}([x : A] B), t, s] = \mathcal{M}[\Gamma; s]^* (\text{eval}_{\mathcal{M}[\Gamma, x : A; B]} \circ p(\mathcal{M}[\Gamma; A])^* \mathcal{M}[\Gamma; t])$  provided that  $\mathcal{M}[\Gamma; t]$ ,  $\mathcal{M}[\Gamma; s]$ ,  $\mathcal{M}[\Gamma, x : A; B]$  are defined, and  $\mathcal{M}[\Gamma; s]$  is a section of  $\mathcal{M}[\Gamma; A]$  and  $\mathcal{M}[\Gamma; t]$  is a section of  $\prod(\mathcal{M}[\Gamma, x : A; B])$



9.  $\mathcal{M}[\Gamma; (\forall x: A)p] = s$  provided  $\mathcal{M}[\Gamma, x: A; p]$  is a section of  $p(\mathcal{M}[\Gamma; A], 1)^* \text{Prop}$  and  $s$  is the unique section of  $p(\mathcal{M}[\Gamma], 1)$  such that

$$\forall (q(p(\mathcal{M}[\Gamma; A], 1), \text{Prop}) \circ \mathcal{M}[\Gamma, x: A; p]) = q(p(\mathcal{M}[\Gamma; A], 1), \text{Prop}) \circ \mathcal{M}[\Gamma, x: A; p]$$



**Theorem 2.11** (Correctness Theorem). *for any doctrine of constructions the following correctness conditions for  $\mathcal{M}$  are satisfied:*

1. if  $\Gamma \text{ ok}$  is derivable, then  $\mathcal{M}[\Gamma]$  is defined;
2. if  $\Gamma \vdash A \text{ type}$  is derivable, then  $\mathcal{M}[\Gamma; A]$  is defined;
3. if  $\Gamma \vdash t: A$  is derivable, then  $\mathcal{M}[\Gamma; A]$  and  $\mathcal{M}[\Gamma; t]$  are both defined and  $\mathcal{M}[\Gamma; t]$  is a section of  $\mathcal{M}[\Gamma; A]$ ;
4. if  $\Gamma = \Delta$  is derivable, then  $\mathcal{M}[\Gamma]$  and  $\mathcal{M}[\Delta]$  are defined and  $\mathcal{M}[\Gamma] = \mathcal{M}[\Delta]$ ;
5. if  $\Gamma \vdash A = B$  is derivable, then  $\mathcal{M}[\Gamma; A]$  and  $\mathcal{M}[\Gamma; B]$  are both defined and  $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Gamma; B]$ ;
6. if  $\Gamma \vdash t = s \in A$  is derivable, then  $\mathcal{M}[\Gamma; A]$ ,  $\mathcal{M}[\Gamma; t]$  and  $\mathcal{M}[\Gamma; s]$  are defined, both  $\mathcal{M}[\Gamma; t]$  and  $\mathcal{M}[\Gamma; s]$  are sections of  $\mathcal{M}[\Gamma; A]$  and  $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Gamma; s]$ .

Hence, doctrines of constructions are models of Calculus of Constructions.

## 2.4 The Term Model of Calculus of Constructions

In this section we will construct a doctrine of constructions  $\mathcal{C}_I$  as a *term model* of calculus of constructions such that the interpretation in this model interprets provably well-defined contexts, types and objects, and identifies only those contexts, types or objects which are provably equal up to renaming of the variables bound in contexts.

In more technical terms, that means that for the interpretation function  $\mathcal{M}$  assigning meaning to calculus of constructions in the doctrine of constructions  $\mathcal{C}_I$ , the following requirements must be fulfilled:

1. for all pre-contexts  $\Gamma$  and  $\Delta$ , if  $\mathcal{M}[\Gamma] = \mathcal{M}[\Delta]$ , then  $\Gamma = \Delta$  is derivable;
2. for all pre-contexts  $\Gamma$  and  $\Delta$  and pre-type-expressions  $A$  and  $B$ , if  $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Delta; B]$ , then  $\Gamma = \Delta$  and  $\Gamma \vdash A = B$  are derivable;
3. for all pre-contexts  $\Gamma$  and  $\Delta$ , pre-type-expressions  $A$  and  $B$  and pre-object-expressions  $t$  and  $s$ , if  $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Delta; s]$  is a section of  $\mathcal{M}[\Gamma; A] = [\Delta; B]$ , then  $\Gamma = \Delta$ ,  $\Gamma \vdash A = B$  and  $\Gamma \vdash t = s : A$  are derivable.

In order to be able to identify contexts up to  $\alpha$ -conversion, we will consider a fixed numerable set of variables:  $v_1, \dots, v_n, \dots$ . We will say that a context  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n$  is in  $\alpha$ -**normal form** iff  $x_i \equiv v_i$  for all  $1 \leq i \leq n$ . Moreover, we will say that a pre-context  $\Gamma$  is admissible iff it is in  $\alpha$ -normal form and  $\vdash \Gamma \text{ok}$ .

Now we will define  $\mathcal{C}_I$ :

The objects of  $\mathcal{C}_I$  are the admissible contexts modulo provable equivalence, i.e. that we will consider two contexts  $\Gamma$  and  $\Delta$  equal iff  $\Gamma = \Delta$  is derivable. By induction over the structure of derivations, one can see that  $\Gamma = \Delta$  implies that  $\Gamma$  and  $\Delta$  have the same length as pre-contexts, so we can define the level of an object of  $\mathcal{C}_I$  as the length of any of its representatives. We will write  $[\Gamma]$  to represent the class containing  $\Gamma$ . If  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n$  is an admissible context, then we define

$$\text{father}[\Gamma] = [v_1 : A_1, \dots, v_{n-1} : A_{n-1}]$$

that is well defined according to rule CREFL1

If  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n$  and  $\Delta \equiv v_1 : B_1, \dots, v_m : B_m$  are admissible contexts, a morphism from  $[\Gamma]$  to  $[\Delta]$  is given by a  $m$ -tuple  $\langle t_1, \dots, t_m \rangle$  of terms such that for  $i, 1 \leq i \leq m$ ,

$$\Gamma \vdash t_i : B_i[v_1 \mid t_1, \dots, v_{i-1} \mid t_{i-1}]$$

is derivable, if  $\langle s_1, \dots, s_m \rangle$  is another  $m$ -tuple such that for  $i, 1 \leq i \leq m$

$$\Gamma \vdash s_i : B_i[v_1 \mid s_1, \dots, v_{i-1} \mid s_{i-1}]$$

is derivable, then  $\langle t_1, \dots, t_m \rangle$  and  $\langle s_1, \dots, s_m \rangle$  are considered equal iff for  $i, 1 \leq i \leq m$

$$\Gamma \vdash t_i = s_i : B_i[v_1 \mid s_1, \dots, v_{i-1} \mid s_{i-1}]$$

is derivable, according to rules CREFL1 and CREFL2 the definition is independent from the choice of representatives. We write  $[\langle t_1, \dots, t_m \rangle] : [\Gamma] \rightarrow [\Delta]$  for the class of  $m$ -tuples equatable to  $\langle t_1, \dots, t_m \rangle$ . If we consider  $\Theta \equiv v_1 : C_1, \dots, v_k : C_k$  is another admissible context and  $[\langle s_1, \dots, s_k \rangle] : [\Delta] \rightarrow [\Theta]$ , then the composition of  $[\langle t_1, \dots, t_m \rangle]$  with  $[\langle s_1, \dots, s_k \rangle]$  is defined as

$$[\langle s_1, \dots, s_k \rangle] \circ [\langle t_1, \dots, t_m \rangle] = [\langle s_1[v_1 \mid t_1, \dots, v_m \mid t_m], \dots, s_k[v_1 \mid t_1, \dots, v_m \mid t_m] \rangle]$$



If  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n$  is an admissible context with  $n \geq 1$  then the *canonical projection* map  $p([\Gamma]) : [\Gamma] \rightarrow \text{father}[\Gamma]$  is represented by the tuple  $\langle v_1, \dots, v_{n-1} \rangle$ . Let  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n$  and  $\Delta \equiv v_1 : B_1, \dots, v_m : B_m$  admissible contexts and  $t = [\langle t_1, \dots, t_m \rangle] : [\Gamma] \rightarrow [\Delta]$  and  $C$  a pre-well-formed type expression with  $\Delta \vdash C$  type, i.e.  $\Delta, v_{m+1} : C$  derivable is an admissible context, then

$$t^*[\Delta, v_{m+1} : C] = [\Delta, v_{m+1} : C[v_1 \mid t_1, \dots, v_m \mid t_m]]$$

$$q([t], [\Delta, v_{m+1} : C]) = [\langle t_1, \dots, t_m, v_{m+1} \rangle : t^*[\Delta, v_{m+1} : C] \rightarrow [\Delta, v_{m+1} : C]]$$

If  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n, v_{n+1} : B, v_{n+2} : C$  is an admissible context we define

$$\prod([\Gamma]) = [\Gamma, v_{n+1} : (\prod v_{n+1} : B)C]$$

$$ev_{[\Gamma]} = [\langle v_1, \dots, v_n, v_{n+1}, \text{App}([v_{n+1} : B]C, v_{n+2}, v_{n+1}) \rangle]$$

which can be seen to be a morphism from  $[\Gamma, v_{n+1} : B, v_{n+2} : (\prod v_{n+1} B)C]$  to  $[\Gamma]$  over the object  $[v_1 : A_1, \dots, v_n : A_n]$ . The object  $\text{Prop}$  of  $C_I$  is defined as  $[v_1 : \text{Prop}]$  and the object  $T$  of  $C_I$  is defined as the object  $[v_1 : \text{Prop}, v_2 : \text{Proof}(v_1)]$ . If  $\Gamma \equiv v_1 : A_1, \dots, v_n : A_n, v_{n+1} : B$  and  $[p]$  is a morphism from  $[\Gamma]$  to  $[v_1 : \text{Prop}]$  then we define

$$\forall([p]) = [\langle (\forall v_{n+1} : B)p \rangle] : [v_1 : A_1, \dots, v_n : A_n] \rightarrow [v_1 : \text{Prop}]$$

It is proven in [6] that  $C_I$  is in fact a doctrine of constructions. Also, it is immediate to see that  $C_I$  does not interpret or identify more objects or terms than those that are provably defined as equal.

**Theorem 2.12** (Completeness Theorem). *Let  $\mathcal{M}$  be the interpretation of calculus of construction in  $C_I$ , then*

1. *if  $\mathcal{M}[\Gamma]$  is defined then  $\Gamma \text{ok}$  is derivable;*
2. *if  $\mathcal{M}[\Gamma; A]$  is defined then  $\Gamma \vdash A \text{type}$  is derivable;*
3. *if  $\mathcal{M}[\Gamma; t]$  is defined then  $\Gamma \vdash t : A$  is derivable for some pre-type-expression  $A$ ;*
4. *if  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n$ ,  $\Delta \equiv y_1 : B_1, \dots, y_m : B_m$  and  $\mathcal{M}[\Gamma; A] = \mathcal{M}[\Delta; B]$ , then  $n = m$  and*

$$\begin{aligned} v_1 : A_1[x_1 \mid v_1, \dots, x_n \mid v_n], \dots, v_n : A_n[x_1 \mid v_1, \dots, x_n \mid v_n] = \\ = v_1 : B_1[y_1 \mid v_1, \dots, y_m \mid v_m], \dots, v_m : B_m[y_1 \mid v_1, \dots, y_m \mid v_m] \end{aligned}$$

$$\begin{aligned} v_1 : A_1[x_1 \mid v_1, \dots, x_n \mid v_n], \dots, v_n : A_n[x_1 \mid v_1, \dots, x_n \mid v_n] \\ \vdash A[x_1 \mid v_1, \dots, x_n \mid v_n] = B[y_1 \mid v_1, \dots, y_m \mid v_m] \end{aligned}$$

*are derivable;*

5. if  $\Gamma \equiv x_1 : A_1, \dots, x_n : A_n, \Delta \equiv y_1 : B_1, \dots, y_m : B_m$  and  $\mathcal{M}[\Gamma; t] = \mathcal{M}[\Delta, s]$ , then  $n = m$  and

$$\begin{aligned} v_1 : A_1[x_1 \mid v_1, \dots, x_n \mid v_n], \dots, v_n : A_n[x_1 \mid v_1, \dots, x_n \mid v_n] = \\ = v_1 : B_1[y_1 \mid v_1, \dots, y_m \mid v_m], \dots, v_m : B_m[y_1 \mid v_1, \dots, y_m \mid v_m] \end{aligned}$$

is derivable, and for some pre-type-expression  $A$

$$\begin{aligned} v_1 : A_1[x_1 \mid v_1, \dots, x_n \mid v_n], \dots, v_n : A_n[x_1 \mid v_1, \dots, x_n \mid v_n] \\ \vdash t[x_1 \mid v_1, \dots, x_n \mid v_n] = s[y_1 \mid v_1, \dots, y_m \mid v_m] : A \end{aligned}$$

is derivable.

It can be also shown that  $\mathcal{C}_I$  is the initial object of the category of doctrines of constructions and structure preserving functors, named **DoC**. Hence the term model satisfies the properties of soundness, completeness and initiality that we wanted.

## 2.5 Extending Calculus of Constructions and its Interpretation

In this section we will show how to add symbols to Calculus of Constructions, the same thing that we could do in GATs. We need to understand the current CoC as just as a minimal starting point, same as the empty theory for GATs. Inspired by the constructions for Categories with Families presented in this paper [1], we will inductively define the concept of signature  $\Sigma$  (corresponding to theory when talking about GATs), CoC extended by  $\Sigma$ , the corresponding category **DoC** $_{\Sigma}$  and interpretation functions  $\mathcal{M}^{\Sigma}$ .

As a starting case, we will consider the empty signature  $\Sigma = \emptyset$ , together with **CoC** $_{\emptyset}$  representing the current syntax of Calculus of Constructions. The category **DoC** $_{\emptyset}$  is exactly **DoC**, and the notion of interpretation morphism stays the same.

Now, for the inductive step, assume we have defined  $\Sigma$  as a valid signature, and the associated syntax **CoC** $_{\Sigma}$ , category **DoC** $_{\Sigma}$  and interpretation function  $\mathcal{M}$ . Then

- **Adding a sort symbol:** let  $\Gamma$  be a context in **CoC** $_{\Sigma}$ , then we can extend  $\Sigma$  with a sort symbol  $S$  relative to  $\Gamma$ , to obtain the signature  $\Sigma' = (\Sigma, (\Gamma, S))$ . Then, we extend **CoC** $_{\Sigma}$  to obtain **CoC** $_{\Sigma'}$  by adding a production of pre-type-expressions  $E ::= S$  and the inference rule  $\Gamma \vdash S$ . The category **DoC** $_{\Sigma'}$  has as objects pairs  $(\mathcal{C}, S_{\mathcal{C}})$ , where  $\mathcal{C}$  is an object of **DoC** $_{\Sigma}$  and  $\mathcal{M}_{\mathcal{C}}[\Gamma] \triangleleft A_{\mathcal{C}}$ . Finally, we define  $\mathcal{M}'_{\mathcal{C}}$ , for each object  $\mathcal{C}$  of **DoC** $_{\Sigma'}$ , by adding  $\mathcal{M}'_{\mathcal{C}}[\Gamma; S] = S_{\mathcal{C}}$ .

- **Adding an operator symbol:** let  $\Gamma$  be a context and  $A$  be a type under  $\Gamma$  of  $\text{CoC}_\Sigma$ , we can extend  $\Sigma$  with a new operator symbol  $f$  relative to  $\Gamma$  and  $A$  obtaining  $\Sigma' = (\Sigma, (\Gamma, A, f))$ . Then, we extend  $\text{CoC}_\Sigma$  to obtain  $\text{CoC}_{\Sigma'}$  by adding a production of pre-object-expressions  $t ::= f$  and the inference rule  $\Gamma \vdash f : A$ . The category  $\mathbf{DoC}_{\Sigma'}$  has as objects pairs  $(\mathcal{C}, f_{\mathcal{C}})$ , where  $\mathcal{C}$  is an object of  $\mathbf{DoC}_\Sigma$  and  $f_{\mathcal{C}}$  is a section of  $\mathcal{M}_{\mathcal{C}}[\Gamma, x : A]$ . Finally, we define  $\mathcal{M}'_{\mathcal{C}}$ , for each object  $\mathcal{C}$  of  $\mathbf{DoC}_{\Sigma'}$ , by adding  $\mathcal{M}'_{\mathcal{C}}[\Gamma; f] = f_{\mathcal{C}}$ .
- **Adding an equality of types:** let  $\Gamma$  be a context,  $A, A'$  types under  $\Gamma$  of  $\text{CoC}_\Sigma$ , we can extend  $\Sigma$  with a new equation  $A = A'$  relative to  $\Gamma$ , to obtain  $\Sigma' = (\Sigma, (\Gamma, A, A'))$ . Then, we extend  $\text{CoC}_\Sigma$  to obtain  $\text{CoC}_{\Sigma'}$  by adding the inference rule  $\Gamma \vdash A = A'$ . The category  $\mathbf{DoC}_{\Sigma'}$  is a full subcategory of  $\mathbf{DoC}_\Sigma$  containing the objects  $\mathcal{C}$  of  $\mathbf{DoC}_\Sigma$  such that  $\mathcal{M}_{\mathcal{C}}[\Gamma; A] = \mathcal{M}_{\mathcal{C}}[\Gamma; A']$ .  $\mathcal{M}'$  is the restriction of  $\mathcal{M}$  to the objects of  $\mathbf{DoC}_{\Sigma'}$ .
- **Adding an equality of terms:** let  $\Gamma$  be a context,  $A$  a type under  $\Gamma$  and  $a, a'$  terms of type  $A$  under  $\Gamma$  of  $\text{CoC}_\Sigma$ , we can extend  $\Sigma$  with a new equation  $a = a'$  relative to  $\Gamma$  and  $A$ , to obtain  $\Sigma' = (\Sigma, (\Gamma, A, a, a'))$ . Then, we extend  $\text{CoC}_\Sigma$  to obtain  $\text{CoC}_{\Sigma'}$  by adding the inference rule  $\Gamma \vdash a = a' : A$ . The category  $\mathbf{DoC}_{\Sigma'}$  is a full subcategory of  $\mathbf{DoC}_\Sigma$  containing the objects  $\mathcal{C}$  of  $\mathbf{DoC}_\Sigma$  such that  $\mathcal{M}_{\mathcal{C}}[\Gamma; a] = \mathcal{M}_{\mathcal{C}}[\Gamma; a']$ .  $\mathcal{M}'$  is the restriction of  $\mathcal{M}$  to the objects of  $\mathbf{DoC}_{\Sigma'}$ .

Under this construction, we can consider  $\mathcal{C}_\Sigma$  to be the exact same construction as  $\mathcal{C}_I$ , but with  $\text{CoC}_\Sigma$  instead of  $\text{CoC}_\emptyset$ . Following a similar proof that the original proof for  $\mathcal{C}_I$  in [6] we can prove the corresponding *Completeness Theorems* and that  $\mathcal{C}_\Sigma$  is the initial object of  $\mathbf{DoC}_\Sigma$ .

## 2.6 Problem Redefinition

Now, we want to redefine our problem from the previous definition for GATs to our new syntax, then rephrase it to similar categorical terms such that an extended version of the expansion algorithm works.

The problem now can be formalized as follows: given a valid signature of Calculus of Constructions  $\Sigma$  and a derivable sequent  $\Gamma \vdash A \text{ type}$  in  $\text{CoC}_\Sigma$ , we want to find a pre-object-expression  $t$  such that  $\Gamma \vdash t : A$  is a derivable rule in  $\text{CoC}_\Sigma$ .

We can consider this from the point of view of the category  $\mathcal{C}_\Sigma$  introduced in the previous section. This means that our problem is equivalent to finding a section  $s$  of  $\mathcal{M}[\Gamma; A] = [\Gamma, v_{n+1} : A]$  in  $\mathcal{C}_\Sigma$ , assuming  $\Gamma$  is a context of length  $n$  in normal form.

Moreover, we want to use the reduction of our search space we did on the previous case by considering the category over a base object. To do it we need the following result:

**Theorem 2.13.** *Given  $\mathcal{C}$  a doctrine of constructions, and  $A$  an arbitrary object of  $\mathcal{C}$ . Then the **relativization** of  $\mathcal{C}$  to  $A$ , noted as  $\mathcal{C}_A$ , is defined as:*

1.  $\mathcal{C}_A$  is a category whose objects are the collection of objects  $B$  over  $A$  in  $\mathcal{C}$ , i.e. objects  $B$  in  $\mathcal{C}$  such that  $A \leq B$ ;
2. for two object  $B, C$  in  $\mathcal{C}_A$ , a morphism from  $B$  to  $C$  over  $A$ , i.e. a morphism in  $\mathcal{C}_A$ , is a morphism  $f: B \rightarrow C$  in  $\mathcal{C}$  such that  $p(C, A) \circ f = p(B, A)$ , composition of morphisms is inherited from  $\mathcal{C}$ ;
3.  $\text{father}_A(B) = \text{father}(B)$  for  $B > A$  and  $\text{father}_A(A) = \text{father}(A)$ ;
4. for objects  $B > A$  we put  $p_A(B) = p(B): B \rightarrow \text{father}_A(B)$ ;
5. for  $B, C, D$  objects of  $\mathcal{C}$  with  $\text{father}_A(D) = C$  and morphism  $f: B \rightarrow C$  in  $\mathcal{C}_A$  we put

$$f^*A D = f^*D \quad \text{and} \quad q_A(f, D) = q(f, D)$$

6. for  $B$  in  $\mathcal{C}$  with  $\text{level}_A(B) \geq 2$ ,

$$\prod_A(B) = \prod(B) \quad \text{and} \quad \text{eval}_B^A = \text{eval}_B$$

7. if  $T$  is the collection of types of  $\mathcal{C}$ , then

$$T_A = p(A)^*T$$

is the collection of types of  $\mathcal{C}_A$ .

is a doctrine of constructions.

*Proof.* TODO : write proof

□

With this result, and with the notation of  $\Gamma$  subscript introduced in the second chapter, we will reduce our problem of finding a section of  $[\langle x: A \rangle_\Gamma]$  in  $\mathcal{C}_{\Sigma, \Gamma}$  the relativization of  $\mathcal{C}_\Sigma$  to  $\Gamma$ .

## 2.7 Expansion Algorithm

Now we will extend the expansion algorithm from the previous chapter. We will reintroduce the rules in order to match the syntax we are using for Calculus of Constructions. First, consider  $G$  to be a finite subcategory of  $\mathcal{C}_{\Sigma, \Gamma}$  containing:  $[\langle \rangle_{\Gamma}]$ ,  $[\langle \text{Prop} \rangle_{\Gamma}]$ ,  $\langle x : A \rangle_{\Gamma}$  and their projections. Now we will define the expansion rules. Consider a context  $\Delta$ , then

- E1 For each sort symbol in the signature  $\Sigma$ , with an introductory rule  $\Omega \vdash \text{Stype}$ , such that  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$ ,  $\Omega \equiv z_1 : C_1, \dots, z_w : C_w$  and  $G_n$  contains sections

$$\begin{aligned} [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_1 \rangle_{\Gamma}} [\Delta, z_1 : C_1] \\ [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_2 \rangle_{\Gamma}} [\Delta, z_2 : C_2[z_1 \mid t_1]] \\ & \vdots \\ [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_w \rangle_{\Gamma}} [\Delta, z_w : C_w[z_1 \mid t_1, \dots, z_{w-1} \mid t_{w-1}]] \end{aligned}$$

then we add  $[\Delta, y : A[z_1 \mid t_1, \dots, z_w \mid t_w]]$  and its projection to  $G_{n+1}$ .

- E2 if  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$ , then we add the sections

$$\begin{aligned} [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, y_i \rangle_{\Gamma}} [\Delta, y : B_i] \\ [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, x_j \rangle_{\Gamma}} [\Delta, x : A_j] \end{aligned}$$

and their corresponding projections for each  $1 \leq i \leq m$  and each  $1 \leq j \leq n$ .

- E3 For each sort symbol in the signature  $\Sigma$ , with an introductory rule  $\Omega \vdash F : A$ , such that  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$ ,  $\Omega \equiv z_1 : C_1, \dots, z_w : C_w$  and the following sections exist in  $G_n$

$$\begin{aligned} [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_1 \rangle_{\Gamma}} [\Delta, z_1 : C_1] \\ [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_2 \rangle_{\Gamma}} [\Delta, z_2 : C_2[z_1 \mid t_1]] \\ & \vdots \\ [\Delta] & \xrightarrow{\langle y_1, \dots, y_m, t_w \rangle_{\Gamma}} [\Delta, z_w : C_w[z_1 \mid t_1, \dots, z_{w-1} \mid t_{w-1}]] \end{aligned}$$

then we add the section

$$[\Delta] \xrightarrow{\langle y_1, \dots, y_m, F[z_1 \mid t_1, \dots, z_w \mid t_w] \rangle_{\Gamma}} [\Delta, z : A[z_1 \mid t_1, \dots, z_w \mid t_w]]$$

E4 if  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$ , for every morphism

$$[\langle t_1, \dots, t_{m-1} \rangle_\Gamma] : [\Gamma, z_1 : C_1, \dots, z_w : C_w] \rightarrow [\Gamma y_1 : B_1, \dots, y_{m-1} : B_{m-1}]$$

we construct the corresponding pullback:

$$\begin{array}{ccc} \Gamma, z_1 : C_1, \dots, z_w : C_w, y_m : B_m [y_1 \mid t_1, \dots, y_{m-1} \mid t_{m-1}] & \xrightarrow{\langle t_1, \dots, t_{m-1}, y_m \rangle_\Gamma} & \Gamma, y_1 : B_1, \dots, y_m : B_m \\ \downarrow & & \downarrow \\ \Gamma, z_1 : C_1, \dots, z_w : C_w & \xrightarrow{\langle t_1, \dots, t_{m-1} \rangle_\Gamma} & \Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1} \end{array}$$

to  $G_{n+1}$ .

E5 If  $\Delta \equiv \Gamma, v_1 : B_1, \dots, v_m : B_m, v_{m+1} : C, v_{n+2} : D$  we add

$$\prod([\Delta]) = [\Gamma, v_1 : B_1, \dots, v_m : B_m, v_{m+1} : (\prod v_{m+1} : C)D]$$

$$\text{eval}_{[\Delta]} = [\Gamma, v_1, \dots, v_{m+1}, \text{App}([v_{m+1} : C]D, v_{m+2}, v_{m+1})]$$

a morphism from  $[\Gamma, v_1 : A_1, \dots, v_m : B_m, v_{n+1} : C, v_{n+2} : (\prod v_{n+1} : B)C]$  to  $[\Delta]$ , and the corresponding projections.

E6 If  $\Delta \equiv \Gamma, v_1 : B_1, \dots, v_m : B_m, v_{m+1} : C$ , for any morphism  $[\langle p \rangle_\Gamma] : [\Delta] \rightarrow [\langle v_1 : \text{Prop} \rangle_\Gamma]$  we add

$$\begin{aligned} \forall([\langle p \rangle_\Gamma]) &= [\langle (\forall v_{m+1} C) p \rangle_\Gamma] : [\Gamma, v_1 : B_1, \dots, v_m : B_m] \rightarrow [\langle v_1 : \text{Prop} \rangle_\Gamma] \\ &[\Delta, x : \text{Proof}(p)] \end{aligned}$$

and its corresponding projection.

And after each expansion, we complete the category in the following way: for every product object, i.e. for each object  $[\Delta]$  with  $\Delta$  of the form  $\Omega, (\prod x : A)B$ , and every section  $s = \langle x, a \rangle_\Omega$  of  $[\Omega, x : A, y : B]$  we add  $\text{curry}(s) = \langle (\lambda x : A)a \rangle_\Omega$  as a section of  $[\Delta]$ . Also, we add the composition of morphisms to  $G_{n+1}$  in order to complete it as a subcategory of  $\mathcal{C}_{\Sigma, \Gamma}$ .

**Theorem 2.14.** *Every object and morphism of  $\mathcal{C}_{\Sigma, \Gamma}$  is constructible by expansion.*

*Proof.* We will prove this result by induction over the level of contexts, pre-type-expressions and terms.

Consider a context  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$  in  $\text{CoC}_\Sigma$ , and the corresponding object  $[\Delta]$  in  $\mathcal{C}_{\Sigma, \Gamma}$ . We will prove that this object is constructible by expansion. We consider how the sequent  $\Delta \text{ok}$  was derived, it has to be by one of the following rules: **EMPTY**, **CONT-INTRO**, **CONT-THIN**, **CONT-SUB**, **CREFL** and **REFLECTION1**. The case of **CREFL** and **REFLECTION1** are trivial, hence we consider each of the other cases:

**EMPTY** This is the base case, when  $\Delta$  is the empty context. This may only happen when  $\Gamma = \Delta = \langle \rangle$  and  $m = 0$ , and as the object  $[\Gamma]$  is included in  $G$ , it is always constructible by expansion.

**CONT-INTRO** This means that the sequent  $\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1} \vdash A_m \text{type}$  is derivable. Which itself can only be derivable using the following rules: **CONT-INTRO** (in the reverse form), **REFL-TYP**, **[ ]-FORM**, **PROP**, **PROP-TYPE** and any introductory rule of a sort symbol in  $\Sigma$ . The derivation from rules **CONT-INTRO** and **REFL-TYP** are trivial, as they both enforce the existence of the object  $[\Delta]$  by premise, hence we will consider the other cases:

**[ ]-FORM** Then, we have that  $A_m$  is a pre-type-expression of the form  $([ ] x : N)M$  for some variable  $x$  and two pre-type-expressions  $N, M$  such that the sequent  $\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}, x : N \vdash M \text{type}$  is derivable. By induction hypothesis, the object

$$\begin{aligned} \mathcal{M}[\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}, x : N; M] &= \\ &= [\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}, x : N, y : M] \end{aligned}$$

is constructible by expansion. By the expansion rule E5 applied to that object, gives us that

$$\begin{aligned} [ ]([\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}, x : N, y : M]) &= \\ &= [\Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}, x : ([ ] x : N)M] \end{aligned}$$

is constructible, that is equal to  $[\Delta]$  as they just differ by the naming of bound variables. Hence,  $[\Delta]$  is constructible by expansion.

**PROP** Then we have that  $A_m$  is equal to **Prop** as a pre-type-expression, and and, with  $\Delta' \equiv \Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}$ ,  $\Delta' \text{ok}$  is a derivable sequent. By induction hypothesis, we know that  $[\Delta']$  is constructible by expansion. Also, as  $G$  contains  $[\Gamma, x : \text{Prop}]$  and it's projection to  $[\Gamma]$ , by applying the expansion rule E4 to  $[\Gamma, x : \text{Prop}]$  with the projection morphism from  $[\Delta']$  to  $[\Gamma]$ , we get that the following pullback diagram is constructible by expansion:

$$\begin{array}{ccc} [\Delta', x : \text{Prop}] & \xrightarrow{\langle x \rangle_\Gamma} & [\Gamma, x : \text{Prop}] \\ \downarrow & & \downarrow \\ [\Delta'] & \longrightarrow & [\Gamma] \end{array}$$

and  $[\Delta', x : \text{Prop}] = [\Delta]$  as they only differ by the naming of the variables.

**PROP-TYPE** Then we have that there is a pre-object-expression  $p$  such that  $A_m$  equals  $\text{Proof}(p)$  as a pre-type-expression and the sequent  $\Delta' \vdash p : \text{Prop}$ , with  $\Delta' \equiv \Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}$ , is derivable. By induction hypothesis,  $[\Delta']$  is constructible by expansion. Moreover, by applying induction hypothesis,

$$\mathcal{M}[\Delta'; p] = \langle p \rangle_\Gamma$$

is constructible by expansion. Finally, by applying expansion rule E6, we get that  $[\Delta] = [\Delta', x : \text{Proof}(p)]$  is constructible by expansion.

**INTRODUCTORY RULE** Consider a sort symbol  $S$  with the corresponding introductory rule  $\Omega \vdash S$ , with  $\Omega \equiv z_1 : C_1, \dots, z_k : C_k$ , and  $t_1, \dots, t_k$  pre-object-symbols such that  $A_m = S[z_1 \mid t_1, \dots, z_k \mid t_k]$ . Then, for each  $i$ ,  $1 \leq i \leq k$ ,  $\Delta' \vdash t_i : C_i[z_1 \mid t_1, \dots, z_{i-1} \mid t_{i-1}]$  are derivable judgments, where  $\Delta' \equiv \Gamma, y_1 : B_1, \dots, y_{m-1} : B_{m-1}$ . Hence, by inductive hypothesis, the sections

$$\mathcal{M}[\Delta'; t_1] : [\Delta'] \rightarrow [\Delta', z_1 : C_1]$$

$$\mathcal{M}[\Delta'; t_2] : [\Delta'] \rightarrow [\Delta', z_2 : C_2[z_1 \mid t_1]]$$

$\vdots$

$$\mathcal{M}[\Delta'; t_k] : [\Delta'] \rightarrow [\Delta', z_k : C_k[z_1 \mid t_1, \dots, z_{k-1} \mid t_{k-1}]]$$

are constructible by expansion. Then, by expansion rule E1,  $[\Delta] = [\Delta', y : S[z_1 \mid t_1, \dots, z_m \mid t_m]]$  is constructible by expansion.

**CONT-THIN** Since context  $\Delta$  is of the form  $\Gamma, y_1 : B_1, \dots, y_m : B_m$ , then there exists an  $k$ ,  $1 \leq k \leq m$  such that  $\Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1}, y_{k+1} : B_{k+1}, \dots, y_m : B_m$  is a context and  $\Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1} \vdash B_k \text{type}$  is a derivable sequent. By induction hypothesis, if  $\Delta' \equiv \Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1}$ ,  $[\Delta']$  is constructible by expansion, and by the same argument in the last point, so is  $[\Delta', y_k : B_k]$ . Finally, by considering the expansion rule E4 on  $[\Delta', y_{k+1} : B_{k+1}, \dots, y_m : B_m]$  and the projection morphism of  $[\Delta', y_k : B_k]$  onto  $[\Delta']$ , we get that the pullback diagram

$$\begin{array}{ccc} [\Delta] \xrightarrow{\langle y_1, \dots, y_{k-1}, y_{k+1}, \dots, y_m \rangle_\Gamma} [\Delta', y_{k+1} : B_{k+1}, \dots, y_m : B_m] \\ \downarrow \qquad \qquad \qquad \downarrow \\ [\Delta', y_k : B_k] \longrightarrow [\Delta'] \end{array}$$

is constructible by expansion, hence, so is  $[\Delta]$ .



CONT-SUB Then there exists a  $k$ ,  $1 \leq k \leq m$ , a variable  $x$  not appearing in  $\Delta$ , pre-type-expressions  $A, B'_k, \dots, B'_m$  and a pre-object-expression  $t$  such that  $\Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1}, x : A, y_k :$  and  $\Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1} \vdash t : A$  are derivable sequents and  $B_i = B'_i[x \mid t]$  for  $k \leq i \leq m$ . For simplicity of notation, we will note  $\Delta' \equiv \Gamma, y_1 : B_1, \dots, y_{k-1} : B_{k-1}$ . Then, by induction hypothesis we have that both contexts  $[\Delta', x : A]$  and  $[\Delta', x : A, y_k : B'_k, \dots, y_m : B'_m]$  and the section  $[\langle y_1, \dots, y_{k-1}, t \rangle_\Gamma]$  are constructible by expansion. Hence, by rule E4, so is the following pullback diagram:

$$\begin{array}{ccc} [\Delta', y_k : B'_k[x \mid t], \dots, y_m : B'_m[x \mid t]] & \xrightarrow{[\langle s, y_k, \dots, y_m \rangle_{\Delta'}]} & [\Delta', x : A, y_k : B'_k, \dots, y_m : B'_m] \\ \downarrow & & \downarrow \\ [\Delta'] & \xrightarrow{[\langle s \rangle_{\Delta'}]} & [\Delta', x : A] \end{array}$$

Finally,  $[\Delta] = [\Delta', y_k : B'_k[x \mid t], \dots, y_m : B'_m[x \mid t]]$  is constructible by expansion.

Now consider a morphism

$$[\langle t_1, \dots, t_w \rangle_\Gamma] : [\Gamma, y_1 : B_1, \dots, y_m : B_m] \rightarrow [\Gamma, z_1 : C_1, \dots, z_w : C_w]$$

in  $\mathcal{C}_{\Sigma, \Gamma}$ . We will note  $\Delta \equiv \Gamma, y_1 : B_1, \dots, y_m : B_m$ . Then for  $i$ ,  $1 \leq i \leq w$ ,

$$\Delta \vdash t_i : C_i[z_1 \mid t_1, \dots, z_{i-1} \mid t_{i-1}]$$

are derivable sequents of  $\text{CoC}_\Sigma$ . Now we want to see that for each  $i$ , the following section is constructible by expansion:

$$[\langle y_1, \dots, y_m, t_i \rangle_\Gamma] : [\Delta] \rightarrow [\Delta, z_i : C_i[z_1 \mid t_1, \dots, z_{i-1} \mid t_{i-1}]]$$

We will consider the ways the mentioned sequents are derived. They must be derived by one of the following rules: ASS, REFL-OBJ, CONV1,  $\Pi$ -INTRO,  $\Pi$ -ELIM, PROP-TYPE and  $\forall$ -INTRO. The cases of REFL-OBJ and CONV1 are trivial, hence we will consider the other cases:

Ass We have that there exists an  $1 \leq j \leq m$  or  $1 \leq k \leq n$  such that

$$t_i = y_j \quad \text{orange} \quad t_i = x_k$$

we will suppose that  $t_i = y_j$  for some  $j$ , the other case is equivalent. In that case  $B_j = C_i[z_1 \mid t_1, \dots, z_{i-1} \mid t_{i-1}]$ . We know that  $\Delta$  is a context, hence by inductive hypothesis we have that  $[\Delta]$  is constructible by expansion. When applying the expansion rule E2, we get that the section

$$[\langle y_1, \dots, y_m, y_j \rangle_\Gamma] : [\Delta] \rightarrow [\Delta, y : B_j]$$

is constructible by expansion, and it is equal to  $[\langle y_1, \dots, y_m, t_i \rangle_\Gamma]$ , hence the least is also constructible by expansion.

$\prod$ -INTRO Then there exists one pre-object-expression  $t$  and two pre-type-expression  $A, B$  such that  $t_i = (\lambda x: A) t$  and  $C_i[z_1 \mid t_1, \dots, z_{i-1} \mid t_{i-1}] = (\prod x: A) B$ , for some variable  $x$ . Moreover, as  $\text{level}[\Delta; (\lambda x: A) t] = \text{level}[\Delta, x: A; t] + 1$ , by induction hypothesis, the section

$$[\langle y_1, \dots, y_m, x, t \rangle_\Gamma]: [\Delta, x: A] \rightarrow [\Delta, x: A, y: B]$$

is constructible by expansion. Applying the expansion rule E5 to  $[\Delta, x: A, y: B]$ , we get that  $[\Delta, f: (\prod x: A) B]$  is constructible by expansion. Finally by the curry completion condition we get that

$$\text{curry}(\langle y_1, \dots, y_m, x, t \rangle_\Gamma) = \langle y_1, \dots, y_m, (\lambda x: A) t \rangle_\Gamma$$

is constructible by expansion.

$\prod$ -ELIM Then there exists one variable  $x$ , two pre-type-expressions  $A, B$  and two pre-term-expressions  $t, s$  such that  $\Delta' \vdash t: (\prod x: A) B$ ,  $\Delta' \vdash s: A$ ,  $\Delta', x: A \vdash B \text{ type}$  and  $B_m = \text{App}([x: A] B, t, s)$ . TODO: finish this, photo of partial proof

PROP-TYPE TODO

$\forall$ -INTRO TODO

□

## Chapter 3

# Implementation of the Expansion Algorithm

In this chapter we will explain the basic ideas on how to use the expansion algorithm jointly with graph neural networks with the objective of generating terms of a specific type in a given theory. We will also present a demo of the raw expansion algorithm for generating terms on simply typed combinatory logic in order to give some intuition on how the algorithm works.

### 3.1 Term generation in Simply Typed Combinatory Logic

Now we will present the `IMPLICA` package for python [5], a graph engine built on rust that we built for the propose of this demonstration. The idea of this package is to allocate the finite subcategory  $G$ , and help to apply the different expansion iterations via its cypher-like interface directly from python.

First we will start by stating what simply typed Combinatory Logic, or  $\mathbf{CL}_{\rightarrow}$ , is. Given a countable set of type variables  $P_0, P_1, P_2, \dots$ , we define the set of simple types  $\mathcal{T}_{\rightarrow}$  as the set generated by the following clauses:

1. type variables belong to  $\mathcal{T}_{\rightarrow}$ ;
2. if  $A, B \in \mathcal{T}_{\rightarrow}$ , then  $(A \rightarrow B) \in \mathcal{T}_{\rightarrow}$ .

Types of the form  $A \rightarrow B$  are called *function types*, and are the simply typed version of the product types we introduced in CoC. Terms of  $\mathbf{CL}_{\rightarrow}$  must have a type assigned and are generated by the following clauses:

1. for each  $A \in \mathcal{T}_{\rightarrow}$  there is a countably infinite supply of variables of type  $A$ ;

2. if  $t, s$  are terms of types  $A \rightarrow B$  and  $A$  respectively, then  $\text{App}(t, s)$  is a term of type  $B$ ;
3. for all  $A, B, C \in \mathcal{T}_{\rightarrow}$  there are constant terms

$$\mathbf{k}^{A,B}: A \rightarrow B \rightarrow A$$

$$\mathbf{s}^{A,B,C}: (A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$$

We will use the following notation: to assert that  $t$  is a term of type  $A$ , we will write  $t: A$ ; instead of writing  $\text{App}(t, s)$  we can simply write  $ts$ .

We can write this as a GAT signature as follows:

$$\begin{aligned} & \vdash \text{Proptype} \\ & p: \text{Prop} \vdash \text{Proof}(p) \text{type} \\ & A, B: \text{Prop} \vdash (A \rightarrow B): \text{Prop} \\ & A, B: \text{Prop}, t: \text{Proof}(A \rightarrow B), s: \text{Proof}(A) \vdash \text{App}(t, s): \text{Proof}(B) \\ & A, B: \text{Prop} \vdash \mathbf{k}^{A,B}: \text{Proof}(A \rightarrow B \rightarrow A) \\ & \text{begin align} * A, B, C: \text{Prop} \vdash \mathbf{s}^{A,B,C}: \text{Proof}((A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C) \end{aligned}$$

This will let us leverage the construction of  $C(U)$  we made for the general case, but this will be rather difficult to follow. Hence, we will look for simplification both of the category and the expansion rules. Consider  $\Gamma$  a simple type expression of  $\mathbf{CL}_{\rightarrow}$ , let  $A_1, \dots, A_n$  be the type variables appearing on  $\Gamma$ , and let  $\Delta$  be the context  $A_1: \text{Prop}, \dots, A_n: \text{Prop}$ . We center our search on the relativization of  $C(U)$  by  $\Delta$ ,  $C_{\Delta}$ . This is a contextual category, with all the tree structure, but this tree structure carries little-to-none meaning as we are working with "non-dependent" types, so by just considering objects of the first level and the final object we have all the possible types in  $A_1, \dots, A_n$  and their relevant morphisms. Moreover, the object  $[\Delta, x: \text{Prop}]$  lies in the level 1, but it carries no important information in our case, as its just used for object formation by the general expansion algorithm, but that will be done differently in this special case, so we will remove it. So we can consider  $\mathcal{C}$  to be the full subcategory of  $C(U)$  containing  $[\Delta]$  and  $[\Delta, x: \text{Proof}(p)]$  for each term  $p$  of type  $\text{Prop}$  under context  $\Delta$ . The objects of this category are in 1-1-correspondence with the elements of  $\mathcal{T}_{\rightarrow} \cup \{1\}$ , hence we will note

$$1 := [\Delta] \quad \text{and} \quad p := [\Delta, x: \text{Proof}(p)]$$

In the case of morphisms, we will note them by the term appearing in the  $n + 1$  component, i.e. each morphism  $\langle A_1, \dots, A_n, s \rangle$  in  $\mathcal{C}$  will be noted as just  $s$ . With this notation in mind we define the following rules of expansion: let  $p$  be an element of  $\mathcal{T}_{\rightarrow}$ ,

1. if  $p$  is of the form  $B \rightarrow A$ , we add

$$A \xrightarrow{\mathbf{k}^{A,B}} p$$

2. for each  $q$  in  $G_n$  we add

$$p \xrightarrow{\mathbf{k}^{p,q}} q \rightarrow p$$

3. if  $p$  is of the form  $(B \rightarrow A) \rightarrow A \rightarrow C$ , we add

$$(A \rightarrow B \rightarrow C) \xrightarrow{\mathbf{s}^{A,B,C}} p$$

4. if  $p$  is of the form  $(A \rightarrow B \rightarrow C)$ , we add

$$p \xrightarrow{\mathbf{s}^{A,B,C}} ((A \rightarrow B) \rightarrow A \rightarrow C)$$

Also, when creating an object  $p$ , if  $p$  is of the form  $A \rightarrow B \rightarrow A$ , add the morphism  $\mathbf{k}^{A,B}: 1 \rightarrow p$ , and if  $p$  is of the form  $(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$ , add the morphism  $\mathbf{s}^{A,B,C}: 1 \rightarrow p$ . Then, complete  $G_{n+1}$  with the composition of morphisms in order for it to be a subcategory of  $\mathcal{C}$ . The exhaustivity of this expansion algorithm is easy to check, as it follows directly from the definitions.

Now we will show how to code the expansion algorithm using python and the `IMPlica` package. One peculiarity we developed into `IMPlica` that isn't part of our current definition is the fact that all terms of a type are considered equal, as we just care about having a witness, not the nature of it. In order to add this condition to the GAT signature, it's enough to add the following equation:

$$p: \text{Prop}, t_1, t_2: \text{Proof}(p) \vdash t_1 = t_2$$

But as it is not really part of combinatory logic we left it out of its formal definition, but for our task it is more efficient to consider it this way.

First we will initialize an `implica` graph and we add the proposition  $A \rightarrow A$  using the query API.

```
import implica

# Initialize the graph with Combinatory Logic constants
graph = implica.Graph(
    constants = [
        implica.Constant(("K", "(A:*)->(B:*)->A")),
        implica.Constant("S", "((A:*)->(B:*)->(C:*))->(A->B)->A->C")
    ]
)
```

```
)

# Seed the graph with our target objective (A -> A)
graph.query().create(f"(:{query})").execute()
```

Now, to perform one expansion iteration we execute the following code

```
# Mark all current nodes as 'existed' to differentiate them from new generations
graph.query().match("N:*").set("N", { "existed" : True }).execute()

# Rule 1 & 2: K-Combinator Expansions
(
    self.graph.query()
    # Find existing nodes matching the signature of a K-target
    .match("N:(B:*) -> (A:*) { existed: true }")
    # Create the precursor node and the connecting morphism
    .create("(: A { existed: false })-[::@K(A, B)]->(N)")
    .execute()
)
(
    self.graph.query()
    .match("N:(A:*) { existed: true }")
    # CRITICAL: Limit creation number to prevent memory overflow
    .limit(10)
    .match("M:(B:*) { existed: true }")
    .create("(N)-[::@K(A, B)]->(:B -> A { existed: false })")
    .execute()
)
# Rule 3 & 4: S-Combinator Expansions
(
    self.graph.query()
    .match("N:((A:*) -> (B:*)) -> A -> (C:*) { existed: true }")
    .create("(:A->B->C { existed: false })-[::@S(A, B, C)]->(N)")
    .execute()
)
(
    self.graph.query()
    .match("N:(A:*)->(B:*)->(C:*) { existed: true }")
    .create("(N)-[::@S(A, B, C)]->(:(A -> B) -> A -> C { existed: false })")
    .execute()
)
```

In the first line, we set an attribute "existed" to true in order to distinguish between new nodes and nodes already present at the beginning. It is important to state the *.limit* modifier on the second query, this is because this expansion rule creates, or tries to create a new node for each pair of nodes in the graph, hence it makes complexity explode. To solve this we use the modifier *.limit* to cap the number of creations per original node to 10. Feel free to remove this line. This explosive growth of the complexity of the algorithm is what we will try to solve later by adding graph neural networks.

Let's walk through, iteration by iteration, on the generation of a term of type  $A \rightarrow A$  by the expansion algorithm to get some intuition. For simplicity, we have omitted the 1 element on the graph displays, and also, as we just want witnesses of the existence of a term, we will just have at most one morphism between two objects, and the sections (morphisms from 1) will be displayed simply as red dots instead of blue. The starting point  $G_0$  is rather uninteresting as it just contains the object  $A \rightarrow A$  with no section. In the figure 3.1a, we can see  $G_1$ , it contains three unpopulated objects  $A$ ,  $A \rightarrow A$  and  $(A \rightarrow A) \rightarrow A \rightarrow A$  from left to right. Then, by applying another expansion we get  $G_2$ , figure 3.1b, where we start to see some objects getting populated, for example  $(A \rightarrow A) \rightarrow A \rightarrow A$  with term **sk**. By the next expansion,  $G_3$  grows significantly, see figure 3.1c, but  $A \rightarrow A$  remains unpopulated. Finally, in  $G_4$   $A \rightarrow A$  gets inhabited by the term **skk**, but as you can see in figure 3.1d the exponential growth of the graph continues.

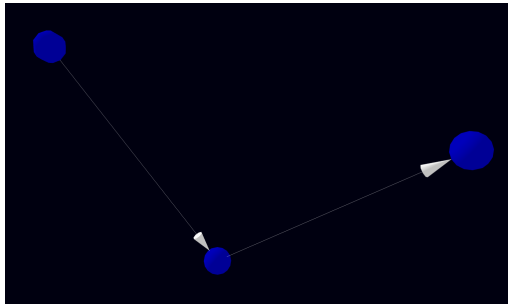
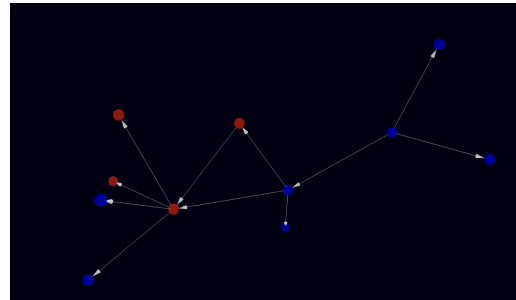
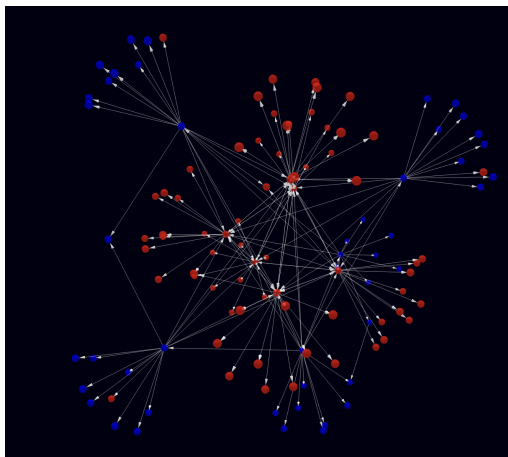
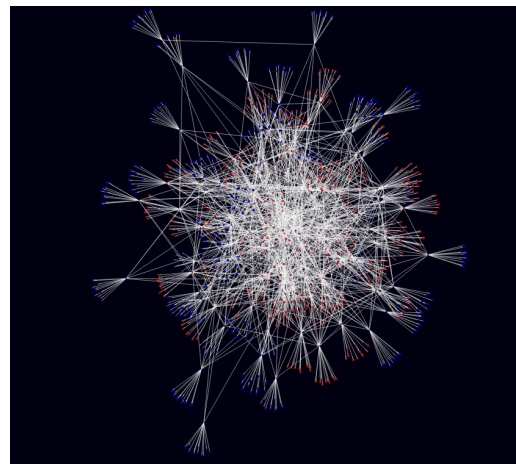
| Iteration ( $G_n$ ) | Nodes (Capped at 10/match) | Nodes (Uncapped) |
|---------------------|----------------------------|------------------|
| $G_0$               | 1                          | 1                |
| $G_1$               | 3                          | 3                |
| $G_2$               | 11                         | 11               |
| $G_3$               | 116                        | 123              |
| $G_4$               | 1352                       | 15131            |

Table 3.1: Node count explosion during the expansion algorithm for the proof of  $A \rightarrow A$  in  $CL_{\rightarrow}$ .

Now that we have a clearer understanding of how the expansion algorithm works, we will introduce the optimizations for the general case.

## 3.2 Graph Neural Networks

Graph Neural Networks (GNNs) process complex data structures by leveraging the relationships between individual data points (nodes) and their connections

(a) The  $G_1$  subcategory(b) The  $G_2$  subcategory(c) The  $G_3$  subcategory(d) The  $G_4$  subcategoryFigure 3.1: Overview of the  $G_1$  through  $G_4$  subcategories.



(edges) through a mechanism known as "message passing". During this process, each node iteratively updates its own representation by gathering and aggregating information from its immediate neighbors, enabling the model to learn both the local features of the data and the broader structural geometry of the network. Because of this unique architecture, GNNs are highly effective for node classification (assigning specific labels to individual nodes), graph classification (categorizing the entire network structure as a single entity), and graph embedding (translating complex graph structures into lower-dimensional vectors for easier processing in downstream machine learning tasks).

The idea behind the expansion algorithm comes from trying to use GNNs to take advantage of the graph structures that emerge from the categorical semantics we presented in the previous chapters. So we built it to work together with a GNN performing node classification. First consider the GNN as just a simple labeler, that given the finite subcategory  $G_n$  produced by the previous expansion, it labels each node in the following 3 categories: KEEP, EXPAND and DELETE. Then we define a partial expansion phase where we compute  $G_{n+1}$  from  $G_n$  without those nodes labeled as DELETE, and by only applying expansion on those nodes marked as EXPAND. This change will make us lose the exhaustivity of the process, but we will be gaining a lot of efficiency if the GNN manages to prune all the useless nodes and expand only those nodes we really need for our generation. In the ideal case, i.e. the GNN always labels nodes correctly, we will be able to still generate terms of any type but we would make it with far less complexity.

Although the implementation of the following algorithm is left for further work, we will present the basic ideas for building our labeling model. The first thing we must note is that using supervised learning techniques (that need to have input - output pairs) need for us to be able to generate a large quantity of training examples by the execution of the raw expansion algorithm, hence we have to discard this option as we would just be able to train our model on very simple examples, due to the high complexity of the algorithm. To solve this we would need to train using unsupervised learning. The technique we are going to use is called Teacher-Student or Conjecturer-Prover. It consists in having a two-headed model, i.e. a model with a GNN based body together with two different projection heads. The first head or conjecturer will propose a challenge, or in our case a type we need to find a term of, and the second head or prover will act as the GNN we talked before and try to find the term with the partial expansion algorithm. Then, if a solution is found, we will compute the optimal labels at each step to compute the loss of the prover head. Then we will compute the loss of the conjecturer by measuring how far it was the prover's loss from some objective loss, this way we make the conjecturer output problems with the right kind of complexity.

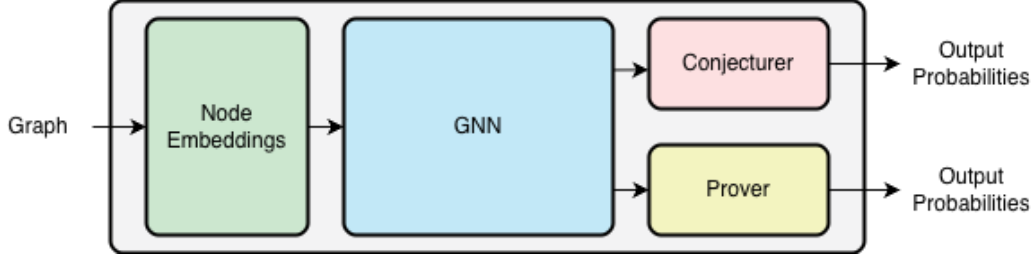


Figure 3.2: Base architecture diagram

We will study the components separately:

**Node Embedding:** The objective of this component is to assign numerical embeddings, i.e. vectors, to the nodes of the input graph. These embeddings represent the raw concept of the node, without taking the graph structure into consideration. Hence it mainly reflects the syntactic meaning of the node. They can be learned or computed from the syntax. If we fix a signature, we could learn basic embeddings for all sort and operator symbols of the signature, assign variables a non-learnable embedding computed just from the name, and learn some embedding merging operations to generate embeddings of more complex expressions or contexts. These merging operations can be performed by small encoder-only transformer models for example. In the case where we have no fixed signature, i.e. we want to create a foundational model to be used with any given signature, we could just learn the aggregation operations, but this is left for further research.

**GNN Body:** The exact architecture of the message-passing GNN has to be determined, but its purpose is to enrich the basic embeddings into structure-aware embeddings. It is important that the architecture selected expects directed edges, as the direction of morphisms is key in this setup.

**Conjecturer:** just a linear projection layer from the original embedding dimension to a 3-dimensional vector. This means that the conjecturer will be classifying nodes into 3 categories: OBJECTIVE, INCLUDE and EXCLUDE; with the constraint that only one node can be chosen to be the objective node, so first we take the node with the highest OBJECTIVE score, mark it as an Objective, then we perform soft-max with the other two components of the other nodes to assign the labels INCLUDE and EXCLUDE. Then the challenge the Prover will try to solve is to generate a section of the OBJECTIVE node starting from the graph composed by the OBJECTIVE node and all the nodes labeled as INCLUDE. Morphisms that exist due to composition or currying of morphisms that include an excluded node won't be included.

**Prover:** this component is just a linear layer from the original embedding dimension to a 3-dimensional vector, same as the conjecturer, and then apply a soft-max. It classifies the nodes in 3 categories: KEEP, EXPAND and DELETE.

Now we have to consider training the model. The first problem is cold start. At the beginning we face an empty graph to feed to the conjecturer and a conjecturer yielding nonsense. This means that we have to somehow pre-train the conjecturer before stating the unsupervised training loop. We will propose a way to go around this problem, but it must undergo testing before assuring it works. We start by feeding the model a type we know it will find a type in relatively few steps of expansion, like  $A \rightarrow A$  in our previous example. We can use this expansion to construct a base graph. This graph contains objects whose sections are fairly easy to construct, as we did it in a few iterations, then we select some of them and use them first as labeled examples for our conjecturer. Then we start the unsupervised training loop. At first, our prover also yields nonsense, so if we let it decide the path of our partial expansion it will fail catastrophically, leaving us with no examples to train our model with. To solve this, we consider a function that determines the score (value) needed by a label in order for us to take action. Name this function  $f: \omega \rightarrow (0, +\infty)$ . As an example, we take the function  $f = 0.8$ , then if we get the output  $(0.3, 0.3, 0.4)$  from the prover, that node would still be label `EXPAND`, although it should be labeled as `DELETE`. In the other hand, if the prover output  $(0.05, 0.1, 0.85)$  for a node, it would be labeled as `DELETE`. A constant function provides little-to-none benefit, because it will cap the complexity of objects our model can handle, as by not allowing drastic pruning the model won't be able to dig to deep into the graph. An example of such a function could be  $f(x) = (a - b) \exp^{-cx} + b$ , with  $a, b, c > 0$ . This confidence baseline  $f(x)$  will decay over time, when  $x$  grows, and it will give more autonomy to the Prover as it learns to accurately slice through the combinatorial explosion.



## **Chapter 4**

## **Conclusion**



# Bibliography

- [1] Marc Bezem, Thierry Coquand, Peter Dybjer, and Martín Escardó, *A Note on Generalized Algebraic Theories and Categories with Families*, March 2021.
- [2] John Cartmell, *Generalized Algebraic Theories and Contextual Categories*, Ph.D. thesis, Oxford University, 1978.
- [3] ———, *Generalised algebraic theories and contextual categories*, *Annals of Pure and Applied Logic* **32** (1986), 209–243.
- [4] Thierry Coquand and Gérard Huet, *The calculus of constructions*, *Information and Computation* **76** (1988), no. 2, 95–120.
- [5] Carlos Fernández Lorán, *Implica at main · CarlosFerLo/implica*.
- [6] Thomas Streicher, *Semantics of Type Theory Correctness, Completeness and Independence Results*, 1991.